DISPENSA · 5ª ITIS INFORMATICA · SISTEMI E RETI

Crittografia: dai cifrari classici a DES e RSA

Un testo completo per studiare senza le slide: dalla scitala e dal cifrario di Cesare fino alla crittografia ibrida di HTTPS e SSH, con tutti gli esempi svolti passo per passo.

COME USARE LA DISPENSA

Ogni capitolo corrisponde a un modulo del percorso (da 0 a 9) e si può leggere da solo; i capitoli si appoggiano uno sull'altro, quindi conviene seguire l'ordine. Nel testo trovi:

- **Esempio svolto**: un calcolo completo con tutti i numeri intermedi, per rifarlo a mano.
- **Da ricordare**: le definizioni e le regole da sapere a memoria.
- **Errore tipico**: gli sbagli più frequenti.
- **Autoverifica**: domande a fine capitolo per controllare di aver capito (senza soluzioni: provale prima su carta).
- **Per esercitarsi**: simulazioni, giochi, schede e laboratori PHP e OpenSSL del percorso.

Indice dei capitoli

0 Introduzione alla crittografia

1 Il cifrario di Cesare

2 Cesare con chiave numerica

3 Il cifrario di Vigenère

4 La trasposizione

5 Cifrari moderni: bit e XOR

6 DES

7 La matematica di RSA

8 RSA

9 Crittografia ibrida, firme e certificati

A Glossario

B Fonti e letture

CAPITOLO 0

Introduzione alla crittografia

Perché si cifra, che cosa vuol dire proteggere un messaggio e quale strada percorreremo: dai cifrari antichi a DES e RSA.

Perché serve la crittografia

Quando Alice invia un messaggio a Bob attraverso una rete, il messaggio non viaggia in linea retta: attraversa il Wi-Fi, gli switch, i router, i dispositivi del provider e infine il server di destinazione. Ognuno di questi nodi è sotto il controllo di qualcun altro. Chi controlla un nodo può leggere il traffico (*sniffing*) oppure modificarlo mentre passa.

Alice, Bob ed Eve sono i nomi convenzionali dei personaggi: Alice è il mittente, Bob il destinatario, Eve (da *eavesdropper*, «chi origlia») è l'attaccante che intercetta. Li useremo in tutta la dispensa. La contromisura è rendere il messaggio incomprensibile a chiunque non possieda il segreto giusto: questa è la **crittografia**. La parola significa «scrittura nascosta» e viene dal greco *kryptós* (nascosto) e *gráphein* (scrivere).

I quattro obiettivi

Un sistema di comunicazione sicuro deve garantire quattro proprietà. La cifratura da sola ne garantisce soltanto una: per le altre servono strumenti diversi, che incontreremo più avanti.

Obiettivo	Che cosa garantisce	Strumento tipico	Esempio
Riservatezza	Solo il destinatario legge il messaggio	Cifratura (capitoli 1-9)	Una chat
Integrità	Il messaggio non è stato modificato	Funzioni hash, codici MAC	Un file scaricato identico all'originale
Autenticazione	Il mittente è chi dice di essere	Password, certificati, firma	Accesso a un sito di home banking
Non ripudio	Il mittente non può negare di aver inviato il messaggio	Firma digitale (capitolo 8)	Un contratto firmato

In questo percorso ci concentriamo sulla riservatezza e sulla firma; l'integrità e le funzioni hash compaiono nei capitoli 8 e 9.

Il lessico di base

Questi termini ricorrono in ogni capitolo. La tabella usa come esempio il cifrario di Cesare con chiave 3, che studieremo nel capitolo 1.

Termine	Significato	Esempio
Testo in chiaro	Il messaggio originale, leggibile	ATTACCO
Testo cifrato	Il messaggio trasformato, illeggibile	DWWDFFR
Algoritmo (cifrario)	La regola con cui si trasforma il testo	spostare le lettere
Chiave	Il dato segreto che governa l'algoritmo	3
Cifratura / decifratura	Da chiaro a cifrato / da cifrato a chiaro	ATTACCO ⇄ DWWDFFR
Crittoanalisi	Leggere o rompere il cifrato senza la chiave	provare tutte le chiavi

> **ERRORE TIPICO**
> In italiano corretto si dice **cifrare** e **decifrare**. «Criptare» è un calco dall'inglese *to encrypt* e si trova in molti testi, ma è meglio evitarlo. Inoltre non si deve confondere *decifrare* (operazione di chi conosce la chiave) con *rompere* o *forzare* un cifrario (crittoanalisi, senza la chiave).

Crittografia, crittoanalisi, steganografia

- **Crittografia**: rende il messaggio *illeggibile*. Tutti sanno che un messaggio c'è, nessuno lo capisce.
- **Crittoanalisi**: cerca di *rompere* il cifrario senza la chiave. È il mestiere di Eve. Insieme, crittografia e crittoanalisi formano la crittologia: ogni cifrario nuovo nasce da un attacco riuscito al precedente.
- **Steganografia**: nasconde l'*esistenza* del messaggio, per esempio dentro un'immagine. Si può combinare con la crittografia (prima si cifra, poi si nasconde), ma è un'altra cosa.

Il principio di Kerckhoffs

> **PRINCIPIO DI KERCKHOFFS**
> La sicurezza di un cifrario deve dipendere **solo dalla segretezza della chiave**, non da quella dell'algoritmo. Formulato da Auguste Kerckhoffs nel 1883.

Le ragioni sono tre. Un algoritmo segreto prima o poi trapela (smontando un dispositivo, analizzando un programma, grazie a una spia). Un algoritmo pubblico, invece, viene esaminato da migliaia di esperti, e i suoi difetti emergono. Infine cambiare una chiave è facile, cambiare un algoritmo no. L'idea opposta si chiama *sicurezza per oscurità* ed è considerata una cattiva pratica. DES e RSA sono pubblici da decenni e restano (con i limiti che vedremo) sicuri: nel corso gli algoritmi saranno sempre noti e l'unico segreto sarà la chiave.

Due famiglie di cifrari: sostituire e mescolare

Tutti i cifrari classici sono combinazioni di due idee elementari. Anche i cifrari moderni lo sono: la *confusione* corrisponde alla sostituzione e la *diffusione* alla trasposizione (capitolo 5).

- **Sostituzione**: ogni lettera diventa un'altra lettera. Le lettere cambiano, la posizione resta. Con il cifrario di Cesare (chiave 3): `ATTACCO` diventa `DWWDFFR`.
- **Trasposizione**: le lettere restano le stesse, cambia l'ordine. È il principio della scitala spartana, che vedremo nel capitolo 4.

Esempio svolto: trasposizione su quattro colonne

Si scrive il messaggio ATTACCO ALL ALBA (senza spazi) su una griglia di 4 colonne, riempiendo per righe, e poi si legge per colonne da sinistra a destra.

A	T	T	A
C	C	O	A
L	L	A	L
B	A	X	X

Il testo cifrato è `ACLBTCLATOAXAALX`. Sono le stesse lettere del messaggio, in un altro ordine; le X finali completano la griglia.

Breve storia

Quando	Evento	Famiglia
V sec. a.C.	Scitala spartana: una striscia di cuoio avvolta su un bastone, descritta da autori greci tra cui Plutarco	trasposizione
I sec. a.C.	Giulio Cesare sposta ogni lettera di tre posti (racconta Svetonio)	sostituzione
1553 - 1586	Bellaso descrive il cifrario con parola chiave; Vigenère ne pubblica una versione. Kasiski ne trova il punto debole nel 1863	sostituzione polialfabetica
1918 - 1945	Enigma, macchina a rotori brevettata da Scherbius, rotta da matematici polacchi e dal gruppo di Bletchley Park (con Alan Turing)	macchina a rotori
1977	DES, standard americano basato su un progetto IBM; sostituito da AES nel 2001	simmetrico
1976 - 1977	Diffie e Hellman propongono la coppia di chiavi; Rivest, Shamir e Adleman realizzano RSA	asimmetrico

Il filo conduttore è che ogni cifrario nasce dall'attacco al precedente: Cesare cade con le frequenze, Vigenère con Kasiski, Enigma con la matematica e le macchine, DES con la potenza di calcolo. RSA nasce da un'idea diversa: la coppia di chiavi.

Crittografia simmetrica e asimmetrica

Una chiave sola

Nella crittografia **simmetrica** Alice e Bob usano la stessa chiave K: Alice cifra con K, Eve vede passare solo il testo cifrato, Bob decifra con la stessa K. Sono simmetrici tutti i cifrari dei capitoli 1-6: Cesare, Vigenère, trasposizione, DES, AES. Il pregio è la velocità, adatta a grandi quantità di dati. Il problema è che Alice e Bob devono già condividere K: come se la scambiano, senza che Eve la veda?

Due chiavi

Nella crittografia **asimmetrica** ogni persona ha una coppia di chiavi legate matematicamente: una **pubblica**, che si può diffondere, e una **privata**, che non esce mai dal suo proprietario. Ciò che una chiave cifra, solo l'altra può decifrare.

- **Riservatezza**: Alice cifra con la chiave *pubblica* di Bob; solo la *privata* di Bob può decifrare.
- **Firma digitale**: Alice firma con la sua chiave *privata*; chiunque verifica con la sua *pubblica*.

	Simmetrica	Asimmetrica
Chiavi	Una sola, segreta e condivisa	Coppia: pubblica e privata
Velocità	Alta	Molto più bassa
Problema	Come scambiare la chiave	Come essere certi di chi è la chiave pubblica
Chiavi per n persone	n(n−1)/2	2n
Esempi	DES, AES	RSA
Uso tipico	Cifrare i dati	Scambiare chiavi, firmare

Quante chiavi servono

Se n persone vogliono parlarsi a coppie con la crittografia simmetrica, ogni coppia ha bisogno di una chiave sua, scambiata in segreto: in tutto n(n−1)/2 chiavi, cioè un numero che cresce con il quadrato di n. Con la crittografia asimmetrica ogni persona ha due chiavi e ne pubblica una: in tutto 2n.

Persone	Chiavi simmetriche	Chiavi asimmetriche
5	10	10
10	45	20
28	378	56
100	4950	200

Una classe di 28 studenti corrisponde alla terza riga.

> **DA RICORDARE**
> Nella pratica i due mondi si usano insieme: la crittografia asimmetrica scambia una chiave segreta, quella simmetrica cifra i dati. È la **crittografia ibrida** (capitolo 9), alla base di HTTPS e SSH.

Un'analogia: la scatola con il doppio lucchetto

Alice vuole far arrivare un biglietto a Bob. Hanno una scatola e due lucchetti con chiavi diverse: la chiave del lucchetto A ce l'ha solo Alice, quella del lucchetto B solo Bob. Non si scambiano mai le chiavi, ed Eve può toccare la scatola ma non aprirla. La soluzione richiede tre viaggi:

1. Alice mette il biglietto nella scatola, la chiude con il lucchetto A e la spedisce a Bob.
2. Bob non può aprire A: aggiunge il proprio lucchetto B e rispedisce la scatola ad Alice (ora ha due lucchetti).
3. Alice toglie il suo lucchetto A e rispedisce la scatola a Bob, che ora ha solo B. Bob lo toglie con la sua chiave, apre e legge.

Nessuna chiave è mai passata da una mano all'altra. Funziona perché i lucchetti si possono togliere in qualunque ordine (in matematica si dice che le operazioni sono *commutative*). Non è però RSA: con RSA Alice usa direttamente il lucchetto «aperto» di Bob, la sua chiave pubblica, e la scatola viaggia una volta

sola. Il punto debole del gioco è che Eve potrebbe fingersi Bob e metterci un lucchetto suo: è il problema dell'autenticazione, che i certificati risolveranno nel capitolo 9.

La mappa del percorso

Capitolo	Argomento	Idea chiave
1	Cifrario di Cesare	Spostare le lettere: il più antico dei cifrari
2	Cesare con chiave numerica	Una serie di spostamenti al posto di uno
3	Vigenère	La chiave diventa una parola
4	Trasposizione	Mescolare le lettere senza cambiarle
5	Cifrari moderni	Bit, XOR, confusione e diffusione
6	DES	Rete di Feistel, S-DES, 3DES e AES
7	Matematica di RSA	Modulo, MCD, inverso, potenze modulari
8	RSA	Chiavi pubblica e privata, firma
9	Crittografia ibrida	Simmetrica e asimmetrica insieme: TLS e SSH

Tre blocchi: i cifrari **classici** (capitoli 1-4, con i relativi attacchi), i cifrari **moderni** (5-6) e quelli **asimmetrici** (7-9). Dopo i capitoli trovi il glossario e le fonti.

Autoverifica

1. Elenca i quattro obiettivi della crittografia e associa a ciascuno uno strumento.
2. Che cosa afferma il principio di Kerckhoffs? Perché un algoritmo pubblico può essere più affidabile di uno segreto?
3. Qual è la differenza tra crittografia, crittoanalisi e steganografia?
4. Cifra la parola SCUOLA con il cifrario di Cesare, chiave 3. Quale famiglia di cifrari è?
5. Dieci persone vogliono comunicare a coppie con la crittografia simmetrica: quante chiavi servono? E con quella asimmetrica?
6. Nel gioco della scatola, perché Alice e Bob non hanno bisogno di scambiarsi le chiavi? Quale attacco rimane possibile?

Per esercitarsi

- Slide del modulo 0 — la presentazione di questo capitolo
- Simulazione: la scatola con il doppio lucchetto — anche da smartphone

CAPITOLO 1

Il cifrario di Cesare

Il più antico dei cifrari a sostituzione: come funziona, come si programma e perché si rompe in pochi minuti.

Idea e storia

Lo storico romano Svetonio racconta che Giulio Cesare, nelle lettere riservate, **sostituiva ogni lettera con quella tre posti più avanti** nell'alfabeto: A diventa D, B diventa E, C diventa F e così via; dopo la Z si riparte dalla A. Svetonio attribuisce ad Augusto una versione con spostamento di un solo posto. Lo spostamento è la **chiave** del cifrario: per Cesare era 3.

Il cifrario di Cesare è un cifrario a **sostituzione monoalfabetica**: ogni lettera in chiaro ha sempre la stessa lettera cifrata. Il termine tornerà con Vigenère. Il cifrario è il punto di partenza di tutto il percorso perché contiene già le idee che ritroveremo in DES e RSA: un algoritmo, una chiave, un attacco.

Come funziona: due alfabeti affiancati

Per cifrare a mano si scrive l'alfabeto due volte. Il secondo è *scorrevole*: lo si sposta della chiave. Per cifrare una lettera si cerca nella riga di sopra e si scrive quella che sta sotto di lei. Con chiave 3:

Chiaro	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Cifrato	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Sulla X, sulla Y e sulla Z l'alfabeto «torna indietro» e si riparte dalla A: è già l'idea dell'aritmetica modulare, che formalizziamo nel prossimo paragrafo.

Dalle lettere ai numeri: l'aritmetica modulare

Per scrivere il cifrario in forma di calcolo si associa a ogni lettera un numero, partendo da **A = 0** (non da 1: in questo modo il resto della divisione funziona senza correzioni).

Lettera	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Numero	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

Cifrare: **E(x) = (x + k) mod 26** Decifrare: **D(y) = (y − k) mod 26**

«mod 26» significa «il resto della divisione per 26»: è come un orologio con 26 ore. Se la somma supera 25 si sottrae 26 e si riparte da A. Nel capitolo 7 riprenderemo il modulo in modo sistematico, perché è la base di RSA.

Esempio svolto: ATTACCO con chiave 3

Si procede una lettera per volta: si trova il numero della lettera, si somma 3, si riduce modulo 26 e si torna alla lettera.

Lettera	x	x + 3	se oltre la Z: − 26	y	Cifrata
A	0	3	—	3	D
T	19	22	—	22	W
T	19	22	—	22	W
A	0	3	—	3	D
C	2	5	—	5	F
C	2	5	—	5	F
O	14	17	—	17	R

Risultato: `ATTACCO` → **`DWWDFFR`**. Le due T e le due C danno la stessa lettera cifrata (W e F): è una caratteristica di tutti i cifrari monoalfabetici, e ne approfitteremo per attaccarli.

Decifrare e contare le chiavi

Decifrare è lo stesso algoritmo con il segno opposto: si sposta ogni lettera **indietro** di k posti (D = A + 3, quindi A = D − 3). Le chiavi utili sono **25**: con chiave 0 il testo non cambia, con chiave 26 si fa un giro completo e si torna alla chiave 0.

Una curiosità è la chiave 13, detta ROT13: applicata due volte restituisce l'originale, perché 13 + 13 = 26. Per esempio «Ciao» diventa «Pvnb» e, cifrata di nuovo, torna «Ciao».

Il resto con i numeri negativi

> **ERRORE TIPICO**
>
> Decifrando la lettera C (x = 2) con k = 5 si ottiene 2 − 5 = −3. In PHP, come in JavaScript, il resto ha il segno del dividendo: `-3 % 26` dà **`-3`**, non 23. Il valore corretto si ottiene con **`(((-3) % 26) + 26) % 26 = 23`**.

La correzione («si somma 26, poi si fa di nuovo il resto») è la prima cosa da ricordare quando si programma un cifrario: la ritroveremo in tutti i capitoli seguenti.

Il cifrario in PHP

Per cifrare una lettera servono due funzioni di PHP: `ord()` trasforma un carattere nel suo codice numerico (in ASCII A vale 65 e a vale 97), `chr()` fa il percorso inverso. Sottraendo il codice di A la lettera diventa un numero da 0 a 25; si somma la chiave, si fa il resto della divisione per 26 e si riconverte.

Passo	Operazione	Per la lettera X con k = 3
1	`ord('X')`	88
2	`- 65 (posizione nell'alfabeto)`	23
3	`+ k`	26
4	`% 26`	0
5	`+ 65, poi chr()`	'A'

La funzione completa lavora su un intero testo. Tre scelte da notare: la chiave viene normalizzata tra 0 e 25 (anche se negativa o troppo grande), maiuscole e minuscole usano basi diverse (65 e 97), e tutto ciò che non è una lettera (spazi, cifre, punteggiatura, lettere accentate) viene copiato senza cambiarlo.

cesare.php

```
<?php
function cesare(string $testo, int $k): string {
    $k = (($k % 26) + 26) % 26;        // chiave sempre tra 0 e 25
    $risultato = '';
    for ($i = 0; $i < strlen($testo); $i++) {
        $c = $testo[$i];
        if ($c >= 'A' && $c <= 'Z') {
            $risultato .= chr((ord($c) - 65 + $k) % 26 + 65);
        } elseif ($c >= 'a' && $c <= 'z') {
            $risultato .= chr((ord($c) - 97 + $k) % 26 + 97);
        } else {
            $risultato .= $c;           // spazi, cifre, punteggiatura
        }
    }
    return $risultato;
}
```

Attacco 1: la forza bruta

Le chiavi sono solo 25: chi intercetta il messaggio le prova **tutte** e legge quale risultato ha senso. Un computer ci mette una frazione di millisecondo. Il testo cifrato dell'esempio è `Ozxlyt gpctqtnl ot cpet`; la tabella mostra il risultato della decifratura con ciascuna chiave.

Chiave	Testo	Chiave	Testo
1	Nywkxs fobspsmk ns bods	14	Aljxkf sbofcfzx af obqf
2	Mxvjwr enarorlj mr ancr	15	Zkiwje ranebeyw ze nape
3	Lwuivq dmzqnqki lq zmbq	16	Yjhvid qzmdadxv yd mzod
4	Kvthup clypmpjh kp ylap	17	Xiguhc pylczcwu xc lync
5	Jusgto bkxoloig jo xkzo	18	Whftgb oxkbybvt wb kxmb
6	Itrfsn ajwnknhf in wjyn	19	Vgesfa nwjaxaus va jwla
7	Hsqerm zivmjmge hm vixm	20	Ufdrez mvizwztr uz ivkz
8	Grpdql yhulilfd gl uhwl	21	Tecqdy luhyvysq ty hujy
9	Fqocpk xgtkhkec fk tgvk	22	Sdbpcx ktgxuxrp sx gtix
10	Epnboj wfsjgjdb ej sfuj	23	Rcaobw jsfwtwqo rw fshw
11	**Domani verifica di reti**	24	Qbznav irevsvpn qv ergv
12	Cnlzmh udqhehbz ch qdsh	25	Paymzu hqduruom pu dqfu
13	Bmkylg tcpgdgay bg pcrg		

Solo una riga è una frase italiana. Il programma può scegliere da solo confrontando le frequenze delle lettere con quelle dell'italiano (punteggio chi-quadro): con testi molto corti la statistica può sbagliare, l'occhio umano no.

Attacco 2: l'analisi delle frequenze

In italiano alcune lettere sono molto più comuni di altre. Poiché Cesare sposta tutte le lettere della stessa quantità, **il profilo delle frequenze resta uguale, ma traslato**. Se nel testo cifrato la lettera più frequente è, per esempio, la L, è probabile che sia la sostituta della E, e la chiave si legge subito.

Lettera	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Frequenza %	11,7	0,9	4,5	3,7	11,8	0,9	1,6	1,5	11,3	0,0	0,0	6,5	2,5	6,9	9,8	3,0	0,5	6,4	5,0	5,6	3,0	2,1	0,0	0,0	0,0	0,5

Frequenze approssimate delle lettere nell'italiano scritto (valori usati dal motore crypto.js).

Un esempio: il testo di un paragrafo sulla crittografia (264 lettere) viene cifrato con una chiave sconosciuta. Le lettere più frequenti nel cifrato sono:

Posizione	Lettera cifrata	Occorrenze	Percentuale
1	L	35	13,3 %
2	H	33	12,5 %
3	P	30	11,4 %
4	Y	22	8,3 %
5	A	20	7,6 %
6	V	17	6,4 %

In italiano le più frequenti sono E, A, I, O, N, L. Facendo scorrere il profilo italiano finché i picchi coincidono si trova lo spostamento: qui la chiave è `7`. L'attacco funziona bene con testi lunghi; con testi corti i profili sono rumorosi.

Perché Cesare non è sicuro

Cosa funziona	Cosa lo rompe
Semplice da spiegare e da programmare	Solo 25 chiavi: la forza bruta è immediata
Ottimo per capire chiave, algoritmo e attacco	Ogni lettera ha sempre la stessa sostituta
Ha ispirato cifrari più robusti	Le frequenze delle lettere restano visibili

Qui il problema è che le chiavi possibili sono troppo poche: in coerenza con il principio di Kerckhoffs, anche se l'algoritmo è noto la sicurezza dovrebbe essere garantita dalla chiave, ma uno spazio di 25 chiavi non protegge nulla. I capitoli successivi mostrano come rimediare alle due debolezze.

DA RICORDARE

- Cesare sposta ogni lettera di **k posti**: E(x) = (x + k) mod 26, con A = 0.
- Decifrare = spostare indietro. Le chiavi utili sono 25.
- In PHP: `ord()`, `chr()`, `%`, con la correzione per i numeri negativi.
- Si rompe con la **forza bruta** (25 chiavi) o con le **frequenze** (profilo traslato).

Autoverifica

1. Cifra «RINVIO A LUNEDI» con chiave 5, scrivendo per ogni lettera numero, somma e resto.
2. Decifra «WKLV LV D WHVW» sapendo che la chiave è 3.
3. Perché le chiavi utili sono 25 e non 26? Che cosa succede con chiave 29?
4. Calcola in PHP (a mano) il valore di −7 % 26 e spiega come ottenere il resto matematico corretto.
5. Spiega perché l'analisi delle frequenze funziona contro Cesare ma non basta a trovare la chiave di un testo di sole 6 lettere.
6. Un cifrato di Cesare ha come lettera più frequente la Q. Quale chiave è plausibile? Su quale ipotesi si basa la risposta?

Per esercitarsi

- Slide del modulo 1
- Simulazione: la ruota di Cesare — anche da smartphone
- Gioco: rompi il messaggio di Cesare — forza bruta e frequenze
- Bigliettini di Cesare — da ritagliare e scambiare
- Lab PHP 01: il cifrario di Cesare — completa, scrivi l'inversa, scrivi l'attacco
- Lab PHP 05: analisi delle frequenze

CAPITOLO 2

Cesare con chiave numerica

Una serie di numeri al posto di uno solo: la chiave si allunga e le frequenze delle lettere si confondono.

Un solo numero è poco

Il cifrario di Cesare ha due punti deboli: ha solo 25 chiavi, che si provano tutte, e ogni lettera ha sempre la stessa sostituta, per cui le frequenze restano visibili. L'idea più naturale per rimediare è cambiare lo spostamento **a ogni lettera**, seguendo una serie di numeri che si ripete.

Con la chiave **3-1-4** la prima lettera si sposta di 3, la seconda di 1, la terza di 4; poi si ricomincia dal primo numero: 3, 1, 4, 3, 1, 4, 3… finché c'è testo. Se la serie ha un solo numero si ottiene di nuovo il Cesare semplice: è un caso particolare. (Non va confuso con il cifrario affine, E(x) = a·x + b, una variante matematica che non fa parte di questo percorso.)

Esempio svolto: ATTACCO ALBA con 3-1-4

La tabella elenca le lettere una per volta. Lo spostamento della *i*-esima lettera è il numero *i* mod 3 della chiave; la colonna «Numero» indica quale numero della chiave si usa.

Lettera	x	Numero della chiave	Spostamento	x + sposta.	Dopo il mod 26	Cifrata
A	0	1	3	3	3	D
T	19	2	1	20	20	U
T	19	3	4	23	23	X
A	0	1	3	3	3	D
C	2	2	1	3	3	D
C	2	3	4	6	6	G
O	14	1	3	17	17	R
A	0	2	1	1	1	B
L	11	3	4	15	15	P
B	1	1	3	4	4	E
A	0	2	1	1	1	B

Risultato: `ATTACCO ALBA` → **`DUXDDGR BPEB`**. Le due T hanno spostamenti diversi (la prima 1, la seconda 4) e quindi danno lettere diverse: è il punto centrale del cifrario. Le due A di ATTACCO, invece, cadono entrambe sul numero 3 e danno la stessa lettera D, perché la lunghezza della chiave è 3 e la quarta lettera ha la stessa posizione nel giro della prima.

Spazi e punteggiatura: la chiave aspetta

> **CONVENZIONE**
> In questo corso la chiave avanza **solo quando si cifra una lettera**. Spazi, cifre, punteggiatura e lettere accentate si copiano e non consumano nessun numero della chiave.

Carattere	C	I	A	O		M	O	N	D	O
Numero usato	1	2	3	1	–	2	3	1	2	3
Spostamento	3	1	4	3	–	1	4	3	1	4
Cifrato	F	J	E	R	␣	N	S	Q	E	S

«CIAO MONDO» diventa `FJER NSQES`. Dopo la O di CIAO (la quarta lettera, numero 1 del secondo giro) lo spazio non fa niente e la M, quinta lettera, usa il numero 2. È la stessa regola di Vigenère: chi riceve può contare le sole lettere. L'alternativa (far avanzare la chiave anche sugli spazi) è possibile, ma va scritta nelle regole: le due convenzioni danno cifrati diversi e due persone che usano convenzioni diverse non riescono a scambiarsi messaggi. È un piccolo esempio di *protocollo* da concordare.

Decifrare: la serie opposta

Decifrare è lo stesso algoritmo con i segni opposti: ogni lettera si sposta indietro di 3, 1, 4, 3, 1, 4… Equivale a spostare in avanti della serie «complementare» `23-25-22`, perché ogni numero s si annulla con 26 − s.

Cifrata	D	U	X	D	D	G	R	B	P	E	B
Spostamento indietro	3	1	4	3	1	4	3	1	4	3	1
In chiaro	A	T	T	A	C	C	O	A	L	B	A

Si ritrova `ATTACCOALBA`.

Stessa lettera, sostituti diversi

Posizione	1	2	3	4	5	6	7
In chiaro	A	T	T	A	C	C	O
Spostamento	3	1	4	3	1	4	3
Cifrato	D	U	X	D	D	G	R

La T diventa U la prima volta e X la seconda; la C diventa D e G. Cambiano i sostituti della stessa lettera, ma solo se cade su numeri diversi: con lo stesso numero la sostituzione è la stessa. Il cifrario non è più monoalfabetico: usa **tre alfabeti**, uno per ogni numero della chiave. Si dice *polialfabetico*.

Il cifrario in PHP

Rispetto al capitolo 1 ci sono due novità: la chiave è un array di interi, e un contatore `$j` conta le lettere già cifrate, così che lo spostamento sia `$chiave[$j % $n]` (indice ciclico). Il ramo `else` usa `continue` perché sugli spazi la chiave non avanza.

cesare-generalizzato.php

```php
<?php
function cesareGeneralizzato(string $testo, array $chiave): string {
    $n = count($chiave);                    // quanti numeri ha la chiave
    if ($n === 0) { return $testo; }        // chiave vuota: niente da fare
    $chiave = array_map(fn($x) => (($x % 26) + 26) % 26, $chiave);
    $risultato = '';
    $j = 0;                                 // quante lettere ho già cifrato
    for ($i = 0; $i < strlen($testo); $i++) {
        $c = $testo[$i];
        if ($c >= 'A' && $c <= 'Z') { $base = 65; }
        elseif ($c >= 'a' && $c <= 'z') { $base = 97; }
        else { $risultato .= $c; continue; }   // non è una lettera: la chiave NON avanza
        $s = $chiave[$j % $n];              // lo spostamento di questa lettera
        $risultato .= chr((ord($c) - $base + $s) % 26 + $base);
        $j++;                               // alla prossima lettera, il numero dopo
    }
    return $risultato;
}

echo cesareGeneralizzato('ATTACCO', [3, 1, 4]);
```

Tre trabocchetti

- **Numeri negativi**: in PHP `-3 % 26` vale −3, non 23. Ogni numero della chiave va normalizzato.
- **Chiave vuota**: `$j % 0` è una divisione per zero (in PHP 8 lancia un'eccezione). Va gestita prima del ciclo.
- **Contare lettere, non caratteri**: se `$j` aumentasse anche sugli spazi, il cifrato cambierebbe e non sarebbe più leggibile con la convenzione del corso. L'errore non produce messaggi: si scopre solo confrontando con un esempio svolto a mano.

L'attacco: quante chiavi?

Ogni numero della chiave ha 26 valori. Con n numeri le chiavi sono 26^n: crescono in modo esponenziale a ogni numero in più. La tabella stima il tempo per provarle tutte supponendo un miliardo di chiavi al secondo.

Numeri nella chiave	Chiavi possibili (26")	Tempo per provarle tutte
1	26	meno di un millisecondo
2	676	meno di un millisecondo
3	17.576	meno di un millisecondo
4	456.976	meno di un millisecondo
6	308.915.776	308,9 millisecondi
8	208.827.064.576	3,5 minuti
10	141.167.095.653.376	2 giorni
12	95.428.956.661.682.176	3 anni

Il conto vale però soltanto per chi prova le chiavi una per una. La struttura del cifrario permette di fare molto meglio.

Le frequenze si appiattiscono

Con più numeri la stessa lettera in chiaro finisce su lettere cifrate diverse, e il profilo delle frequenze del testo intero si appiattisce. L'indice di coincidenza (IC), che misura quanto un testo somiglia all'italiano (circa 0,074) o a lettere casuali (circa 0,038), scende al crescere della lunghezza della chiave. L'attacco del capitolo 1 sul testo intero non funziona più.

Se conosci la lunghezza, ogni colonna è un Cesare

Le lettere nelle posizioni 1, 4, 7… sono state spostate tutte dello stesso numero; lo stesso per le posizioni 2, 5, 8… e 3, 6, 9… Se si conosce la lunghezza *n* della chiave, si divide il testo cifrato in *n* **colonne** e ciascuna è un cifrario di Cesare con la sua chiave, che si rompe con le frequenze.

Esempio svolto: un paragrafo cifrato con una chiave di lunghezza 3

Il testo cifrato (264 lettere, senza spazi) viene diviso in 3 colonne di circa 88 lettere ciascuna. Le lettere più frequenti di ogni colonna e lo spostamento stimato con le frequenze sono:

Colonna	Lettere più frequenti	Spostamento stimato
1	L H D	3
2	B F J	1
3	I E M	4

Le tre colonne forniscono `3-1-4`: la chiave è ricostruita. Con pochi caratteri per colonna la statistica sbaglia, quindi serve un testo abbastanza lungo.

Resta una domanda: come si scopre la lunghezza della chiave? La risposta (Kasiski e indice di coincidenza) è nel capitolo 3. Se invece la chiave è di due numeri le chiavi sono soltanto 676: il computer le prova tutte e tiene quelle che producono una *parola probabile* attesa nel testo (per esempio «bonifico»).

Numeri o lettere: è la stessa chiave

Poiché A vale 0, B vale 1 e così via, una serie di numeri è una parola e viceversa. La chiave `3-1-4` coincide con la chiave `DBE` (D = 3, B = 1, E = 4): ATTACCO diventa `DUXDDGR` con i numeri e `DUXDDGR` con la parola DBE. Il cifrario di Vigenère è esattamente questo, con una parola da ricordare al posto di una serie di numeri; lo studiamo nel capitolo 3.

Lettera	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Numero	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

Debolezze

Cosa migliora	Cosa resta debole
Le chiavi crescono come 26^n	La chiave si ripete: le colonne sono Cesare
Le frequenze del testo intero si appiattiscono	Sequenze ripetute nel testo tradiscono la lunghezza
Ancora semplice da spiegare e programmare	Una chiave di numeri è scomoda da ricordare

Più la chiave è lunga, meglio protegge. Il caso limite è una chiave lunga quanto il messaggio e usata una sola volta (one-time pad, capitoli 3 e 5).

> **DA RICORDARE**
>
> - La chiave è una **serie di numeri**: la *i*-esima lettera si sposta di chiave[*i* mod *n*].
> - La chiave avanza solo sulle lettere. Decifrare = serie complementare (26 − s).
> - Chiavi possibili: 26^n. Ma ogni colonna è un Cesare, e si attacca con le frequenze.
> - In PHP: indice ciclico `$j % $n`.

Autoverifica

1. Cifra «ATTENZIONE» con la chiave 2-5-1, indicando per ogni lettera lo spostamento usato.
2. Qual è la serie complementare della chiave 7-0-19? Come si usa per decifrare?
3. Perché nella chiave 3-1-4 le due T di ATTACCO danno lettere diverse mentre le due A danno la stessa?
4. Quante chiavi ha un cifrario con serie di 6 numeri? Perché questo numero non è l'unico indicatore della sicurezza?
5. Descrivi come si attacca un testo cifrato con chiave di lunghezza nota. Che cosa cambia se la lunghezza non si conosce?
6. Scrivi la chiave in lettere equivalente alla serie 12-20-13-0.

Per esercitarsi

- Slide del modulo 2
- Simulatore e sfida: Cesare a chiave numerica — trova la chiave di un messaggio intercettato
- Bigliettini a chiave numerica — da ritagliare
- Lab PHP 02: Cesare generalizzato — cifra, decifra e attacca con chiave di lunghezza nota

CAPITOLO 3

Il cifrario di Vigenère

Una chiave fatta di lettere: ogni posizione ha il suo spostamento. Per tre secoli sembrò indecifrabile, finché qualcuno guardò le ripetizioni.

Il limite di Cesare e l'idea dei più alfabeti

Con Cesare (e con la sua versione a chiave numerica quando la chiave è corta) una lettera in chiaro ha **sempre** la stessa lettera cifrata. Se la E è la lettera più frequente in italiano, nel testo cifrato lo sarà la sua sostituta, e un occhio allenato la trova. L'idea è usare **più alfabeti** in alternanza, in modo che la stessa lettera in chiaro venga cifrata ogni volta in modo diverso. Un cifrario che usa un solo alfabeto è *monoalfabetico*; con più alfabeti in rotazione è *polialfabetico*.

Un po' di storia

Quando	Evento
1553	Giovan Battista Bellaso pubblica il cifrario con parola chiave
1586	Blaise de Vigenère lo descrive nel «Traicté des chiffres»
XVIII-XIX secolo	Il cifrario è noto come «le chiffre indéchiffrable» (il cifrario indecifrabile)
1854-1863	Charles Babbage lo viola in segreto verso il 1854; Friedrich Kasiski pubblica il metodo nel 1863

Il nome è un equivoco storico: il cifrario con parola chiave è di Bellaso; a Vigenère è rimasto il nome, ma lui descrisse una variante diversa.

Come funziona

La chiave è una **parola**. Ogni sua lettera è uno spostamento: A = 0, B = 1, … Z = 25. La chiave LUNA corrisponde quindi alla serie di spostamenti 11, 20, 13, 0, che si ripete lungo il testo.

Lettera della chiave	L	U	N	A
Spostamento	11	20	13	0

> **RISPETTO A CESARE**
> Cesare è un Vigenère con una chiave di **una sola** lettera: lo spostamento è sempre lo stesso. E una serie di numeri come 3-1-4 è la chiave DBE (capitolo 2).

Cifrare: $C_i = (P_i + K_{i \bmod n}) \bmod 26$ Decifrare: $P_i = (C_i - K_{i \bmod n}) \bmod 26$

Con n lunghezza della chiave, la lettera i usa la lettera numero $i \bmod n$ della chiave: per questo la chiave si ripete.

Esempio svolto: ATTACCO con chiave LUNA

Posizione	Chiaro	x	Lettera chiave	Spostamento	x + sposta.	mod 26	Cifrata
1	A	0	L	11	11	11	L
2	T	19	U	20	39	13	N
3	T	19	N	13	32	6	G
4	A	0	A	0	0	0	A
5	C	2	L	11	13	13	N
6	C	2	U	20	22	22	W
7	O	14	N	13	27	1	B

Risultato: `ATTACCO` → **`LNGANWB`**. Le due T cadono su lettere diverse della chiave (U e N) e danno lettere diverse; la C della posizione 5 e quella della 6 pure. In Cesare tutte le coppie uguali avrebbero dato lo stesso risultato.

Per **decifrare** a mano si sposta ogni lettera cifrata *indietro* della lettera corrispondente della chiave: `LNGANWB` con LUNA dà `ATTACCO`.

La tavola di Vigenère

Per cifrare senza fare calcoli si usa la tavola: la riga della lettera della **chiave** e la colonna della lettera del **testo in chiaro** si incontrano nella lettera cifrata. Per decifrare si cerca la lettera cifrata nella riga della chiave e si legge in cima la colonna. Ogni riga è l'alfabeto spostato di una lettera in più.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Tavola di Vigenère: riga = lettera della chiave, colonna = lettera del chiaro.

Esempio: la T (colonna) cifrata con la chiave U (riga) dà `N`.

Perché è più forte di Cesare

- Il profilo delle frequenze si appiattisce. L'**indice di coincidenza** (IC) misura quanto il profilo è «a picchi»: circa 0,074 per l'italiano, circa 0,038 per lettere casuali. Cesare lo conserva, Vigenère lo abbassa.
- Le chiavi possibili sono 26^n con n lunghezza della chiave: 1 lettere → 26; 4 lettere → 456.976; 8 lettere → 208.827.064.576; 12 lettere → ≈ 9,5 · 10^16.

La formula dell'indice di coincidenza è $IC = \Sigma\, f_i(f_i - 1) / (N\,(N - 1))$, dove f_i è il numero di occorrenze della lettera i e N il numero totale di lettere: è la probabilità che due lettere prese a caso nel testo siano uguali.

Il cifrario in PHP

La funzione è quasi identica a quella di Cesare. La chiave è una stringa: si tolgono i caratteri che non sono lettere, si porta in maiuscolo, e per ogni lettera del testo lo spostamento è `ord($chiave[$j % $lunghezza]) - 65`. Il contatore `$j` aumenta soltanto sulle lettere, così spazi e simboli non consumano lettere della chiave.

vigenere.php

```php
<?php
function vigenere(string $testo, string $chiave): string {
    $chiave = strtoupper(preg_replace('/[^A-Za-z]/', '', $chiave));
    $lunghezza = strlen($chiave);
    $risultato = '';
    $j = 0;                                  // lettere cifrate finora
    for ($i = 0; $i < strlen($testo); $i++) {
        $c = $testo[$i];
        if (ctype_alpha($c)) {
            $base = ctype_upper($c) ? 65 : 97;
            $spostamento = ord($chiave[$j % $lunghezza]) - 65;
            $risultato .= chr((ord($c) - $base + $spostamento) % 26 + $base);
            $j++;                            // la chiave avanza
        } else {
            $risultato .= $c;                // spazi e simboli
        }
    }
    return $risultato;
}
```

L'attacco in tre mosse

Vigenère non è invulnerabile. La sua debolezza sta nel fatto che la chiave **si ripete**. L'attacco procede così:

1. **Stimare la lunghezza** *n* della chiave (Kasiski, indice di coincidenza).
2. **Dividere il testo in *n* colonne**: le lettere nelle posizioni 1, n+1, 2n+1… sono state cifrate tutte con lo stesso spostamento.
3. **Ogni colonna è un cifrario di Cesare**: si rompe con le frequenze. Le lettere trovate formano la chiave.

Come esempio usiamo un paragrafo di 393 lettere cifrato con una chiave sconosciuta. Testo cifrato (a gruppi di 5): `WUPRT NGORL NFTUR LLLGE OCAAD WBNOY EETFF IRHVF TWNTZ XVUYG RSDUT GTIPH TMPRT PRTCU FFZLZ AWYCA CIYEN IAUYU EERIY APOAA NBVAG YPHTF RGRYH SLFNS EYFSL WUILP RPPLG OCHNR PUYTP MGOZL VGTHN LPHRI DYPOW CDUPM GATXR APMGA EUHSL NNPPL CRZNR GRYEE WYGTP LROCX VNTGV LTNNR TYFER LRTTX VSEUG OXUNN NBRPP LTIZW NRPWB NRFVA XCPIN BVRTY FCPUG RZPNR PFNCS CNVPJ HOWYT GPLRT FNGOT FZEDM NGRCB SPHMA NBVEO YEELC HTZUA EDMHN ZOABF IACTZ EACCB NZHFI LZSIO UNLWU FERLR TPTMA OYYLL FTOCC GMZ`

Mossa 1a: il metodo di Kasiski

Se due frammenti uguali del testo in chiaro cadono nella stessa posizione rispetto alla chiave, danno frammenti **uguali** nel testo cifrato. La distanza tra le due ripetizioni è allora un **multiplo** della lunghezza della chiave. Si cercano nel cifrato le sequenze ripetute e si calcola il massimo comun divisore delle distanze.

Sequenza ripetuta	Posizioni (da 0)	Distanza
OYEE	28, 324	296
ANBV	99, 319	220
RGRY	110, 194	84
RPPL	130, 242	112
PMGA	168, 176	8
RTYF	219, 267	48
FERL	222, 370	148
ERLR	223, 371	148
RLRT	224, 372	148

Le distanze sono 296, 220, 84, 112, 8, 48, 148 (le ultime tre righe sono la stessa ripetizione, vista da posizioni sovrapposte): il loro massimo comun divisore è `4`. La lunghezza della chiave è 4, oppure un suo divisore (1 o 2). Le sequenze brevi possono ripetersi per caso: conviene considerare più ripetizioni e usare quelle lunghe.

Mossa 1b: l'indice di coincidenza

Per ogni lunghezza *n* si dividono le colonne e si calcola l'IC medio. Sulle lunghezze giuste (e sui loro multipli) l'IC sale verso il valore dell'italiano, perché ogni colonna è un testo cifrato con un solo spostamento.

Lunghezza n	IC medio
1	0,049
2	0,057
3	0,048
4	**0,077**
5	0,047
6	0,057
7	0,049
8	**0,075**
9	0,046
10	0,055
11	0,049
12	**0,078**

Picchi su 4, 8 e 12 (multipli di 4); per confronto, l'italiano ha IC 0,076 (valore teorico circa 0,074).

Mossa 2 e 3: colonne e frequenze

Con n = 4 il testo si divide in 4 colonne. Per ciascuna si cerca lo spostamento che fa comparire i picchi dell'italiano:

Colonna	Lettere più frequenti	Spostamento stimato	Lettera della chiave
1	P T Z	11	L
2	U Y L	20	U
3	N R G	13	N
4	A E O	0	A

Le quattro lettere formano la chiave `LUNA`. Decifrando il testo si ottiene il paragrafo originale. Una volta nota la lunghezza *n*, il problema diventa *n* volte Cesare.

Come si rafforza: il one-time pad

Se la chiave è lunga quanto il messaggio non c'è nulla da ripetere: niente colonne, niente Kasiski, niente IC. Il caso limite si chiama **one-time pad** («blocco monouso») e garantisce una sicurezza perfetta, a tre condizioni.

Condizione	Significato
Lunga quanto il messaggio	Nessuna ripetizione
Casuale	Ogni lettera uniforme, senza schema (non una parola di senso)
Usata una sola volta	Mai riutilizzata

Perché non si usa dappertutto

Per ogni messaggio serve una chiave nuova, casuale, lunga quanto il messaggio e scambiata in modo sicuro: è lo stesso problema di far arrivare un segreto, ma più grande. Soprattutto, **non si può riusare**. Se due messaggi sono cifrati con la stessa chiave, la differenza dei cifrati coincide con la differenza dei testi in chiaro: la chiave sparisce e il segreto fugge.

Esempio svolto: chiave riusata

Si cifrano `ATTACCO` e `PARTITA` con la stessa chiave `QZKFWBM`. I cifrati sono `QSDFYDA` e `FZBYEUM`. Lettera per lettera, la differenza (mod 26) dei due cifrati è uguale a quella dei due testi in chiaro:

Posizione	1	2	3	4	5	6	7
Chiaro 1	A	T	T	A	C	C	O
Chiaro 2	P	A	R	T	I	T	A
Differenza dei chiari	11	19	2	7	20	9	14
Cifrato 1	Q	S	D	F	Y	D	A
Cifrato 2	F	Z	B	Y	E	U	M
Differenza dei cifrati	11	19	2	7	20	9	14

Chi intercetta i due cifrati ottiene quindi informazioni sui messaggi senza conoscere la chiave. Per questo si chiama «monouso»: non è una raccomandazione, è una condizione.

DA RICORDARE

- Vigenère usa una **parola chiave**: C = (P + K) mod 26, con la chiave che si ripete.
- Si rompe stimando la **lunghezza** (Kasiski, IC) e poi con le **frequenze per colonna**.
- Il caso limite è il **one-time pad**: chiave casuale, lunga quanto il messaggio, usata una volta sola.

ERRORE TIPICO

Riusare la stessa chiave del one-time pad su due messaggi: la differenza dei cifrati coincide con la differenza dei messaggi e la sicurezza perfetta svanisce.

Autoverifica

1. Cifra «INCONTRO ALLE OTTO» con la chiave MARE, scrivendo la chiave sotto il testo (senza far avanzare la chiave sugli spazi).
2. Decifra con la tavola di Vigenère il frammento «PQXL» sapendo che la chiave è LUNA.
3. Perché Vigenère è detto polialfabetico? Quanti alfabeti usa con una chiave di 5 lettere?
4. Descrivi il metodo di Kasiski: che cosa si cerca nel cifrato e che cosa si calcola?
5. Perché, una volta nota la lunghezza n, il problema diventa «n volte Cesare»?
6. Elenca le tre condizioni del one-time pad e spiega che cosa succede se una non è rispettata.

Per esercitarsi

- Slide del modulo 3
- Simulatore di Vigenère — anche da smartphone
- Gioco: rompi il messaggio di Vigenère — Kasiski e frequenze per colonna
- Bigliettini di Vigenère — da ritagliare
- Lab PHP 03: Vigenère — cifra, decifra e attacca

CAPITOLO 4

La trasposizione

Non cambiare le lettere: cambiare l'ordine. Una famiglia di cifrari che ancora oggi è un ingrediente dei cifrari a blocchi.

Sostituzione contro trasposizione

	Sostituzione	Trasposizione
Che cosa cambia	Le lettere; la posizione resta	La posizione; le lettere restano
Esempio	ATTACCO → DWWDFFR (Cesare, chiave 3)	ATTACCO → AXAOCXT… (colonne)
Il cifrato è…	fatto di lettere diverse dal chiaro	un anagramma del chiaro

Se un attaccante vede solo il testo cifrato, su ciascuno dei due tipi ha informazioni diverse: nella sostituzione conosce la posizione ma non le lettere, nella trasposizione conosce le lettere ma non la posizione.

Le frequenze non cambiano

Poiché le lettere sono le stesse, nel cifrato la E, la A, la I compaiono con la stessa frequenza dell'italiano. Lo si vede con l'indice di coincidenza (IC): su un paragrafo di prova, il testo in chiaro e quello trasposto hanno lo stesso IC. Lo conserva anche Cesare, perché sposta tutte le lettere allo stesso modo: l'IC distingue i cifrari monoalfabetici e le trasposizioni da quelli polialfabetici, ma non li distingue tra loro.

Testo	Indice di coincidenza
In chiaro	0,079
Dopo la trasposizione (chiave CHIAVE)	0,079
Dopo Cesare (chiave 7)	0,079

- Un attacco a frequenze **non** dice nulla sulla chiave di una trasposizione: i conteggi sono quelli dell'italiano.
- Ma è anche un indizio: se l'IC è alto e i picchi cadono proprio su E, A, I, O, il cifrato è quasi certamente una **trasposizione**; se l'IC è alto ma i picchi sono su altre lettere, è una sostituzione monoalfabetica (come Cesare).

Dalla scitala a ADFGVX

La **scitala spartana** è una striscia di cuoio avvolta a spirale su un bastone: si scrive nel senso della lunghezza, poi si svolge. Letta srotolata, la striscia mostra lettere in disordine; solo chi ha un bastone dello stesso diametro rimette in ordine il testo. La **trasposizione a colonne** trasforma la striscia in una griglia: il numero di colonne è il «diametro» e la parola chiave decide l'ordine di lettura. Nel 1918 i tedeschi usarono il cifrario **ADFGVX**: una sostituzione seguita da una trasposizione a colonne. Il francese Georges Painvin lo ruppe in pochi mesi.

La trasposizione a colonne

1. Si sceglie una **parola chiave**: la sua lunghezza è il numero di colonne.
2. Si scrive il testo nella griglia **per righe**.
3. Si numerano le colonne secondo l'**ordine alfabetico** delle lettere della chiave.
4. Si leggono le colonne **una alla volta**, in quell'ordine, dall'alto in basso.

La chiave contiene quindi due informazioni: la **larghezza** della griglia e l'**ordine** delle colonne.

Esempio svolto: ATTACCOALLALBADOMANI con la chiave ZEBRA

La chiave ha 5 lettere: la griglia ha 5 colonne. Ordinate alfabeticamente le lettere sono A, B, E, R, Z, quindi la colonna della A (la quinta) si legge per prima, poi la B (terza), la E (seconda), la R (quarta) e infine la Z (prima).

Chiave	Z	E	B	R	A
Turno di lettura	5	3	2	4	1
Riga 1	A	T	T	A	C
Riga 2	C	O	A	L	L
Riga 3	A	L	B	A	D
Riga 4	O	M	A	N	I

Si leggono le colonne dall'alto in basso nell'ordine 1, 2, 3, 4, 5 dei turni:

Turno	Colonna letta	Lettere
1	colonna 5 (A)	CLDI
2	colonna 3 (B)	TABA
3	colonna 2 (E)	TOLM
4	colonna 4 (R)	ALAN
5	colonna 1 (Z)	ACAO

Mettendo in fila le colonne si ottiene `CLDITABATOLMALANACAO`.

Lettere ripetute nella chiave

Se una lettera compare più volte nella chiave, la colonna **più a sinistra** ha la precedenza. Con la chiave `CASA` le due A hanno i turni 1 e 2 (la prima A è la colonna 2, la seconda è la colonna 4).

Chiave	C	A	S	A
Turno di lettura	3	1	4	2
Riga 1	A	T	T	A
Riga 2	C	C	O	A
Riga 3	L	L	A	L
Riga 4	B	A	D	O
Riga 5	M	A	N	I

Ordine di lettura (numeri di colonna): `2, 4, 1, 3`. Il cifrato è **`TCLAAAALOIACLBMTOADN`**.

Quando il testo non riempie la griglia

Il testo `ATTACCOALLALBA` ha 14 lettere e la chiave `LUNA` ha 4 colonne: 14 non è un multiplo di 4, quindi l'ultima riga resta incompleta.

> **CONVENZIONE DI QUESTO CORSO**
> Si riempie sempre con **X** fino a una griglia completa. Decifrando si ottengono le X finali, da riconoscere e ignorare.

Chiave	L	U	N	A
Turno di lettura	2	4	3	1
Riga 1	A	T	T	A
Riga 2	C	C	O	A
Riga 3	L	L	A	L
Riga 4	B	A	X	X

Il cifrato è **`AALXACLBTOAXTCLA`**: 16 lettere, un multiplo di 4.

Decifrare: dal cifrato alla griglia

Chi decifra conosce la chiave (quindi la larghezza e l'ordine) e il numero di lettere. Con 20 lettere e 5 colonne ogni colonna ha 4 lettere. Si procede così:

1. Si taglia il cifrato in 5 pezzi da 4 lettere: `CLDI`, `TABA`, `TOLM`, `ALAN`, `ACAO`.
2. Il primo pezzo appartiene alla colonna con il turno 1 (qui la colonna 5), il secondo alla colonna con il turno 2 (3) e così via.
3. Si rimettono le colonne al loro posto e si legge la griglia **per righe**.

Turno	Pezzo di cifrato	Va nella colonna
1	CLDI	5 (A)
2	TABA	3 (B)
3	TOLM	2 (E)
4	ALAN	4 (R)
5	ACAO	1 (Z)

Leggendo per righe si ottiene `ATTACCOALLALBADOMANI`. Se all'ultima riga compaiono delle X, sono il riempimento.

> **ERRORE TIPICO**
> Contare male le lettere: se la lunghezza del cifrato non è un multiplo del numero di colonne, il testo è stato cifrato con un riempimento diverso. Con la convenzione del corso (sempre X fino alla griglia completa) la lunghezza è sempre un multiplo, e tutte le colonne hanno la stessa altezza.

Combinare i cifrari

Doppia trasposizione

Si ripete la trasposizione sul risultato, con la stessa chiave o con un'altra: le colonne si mescolano due volte. Due permutazioni in sequenza si compongono in una sola permutazione più complessa dell'intero testo. Con ZEBRA due volte `ATTACCOALLALBADOMANI` diventa `TOAODAACLBMAITLACALN`; con ZEBRA e poi LUNA diventa `IAMNOCTTAADBLAALAOLC`.

Sostituzione più trasposizione

Si può prima sostituire e poi mescolare. `ATTACCOALLALBADOMANI` cifrato con Vigenère (chiave LUNA) dà `LNGANWBAWFNLMUQOXUAI`, e trasposto con ZEBRA diventa `NFQIGAMUNBLXAWUALWNO`.

> **PONTE VERSO I CIFRARI MODERNI**
> Claude Shannon (1949) indicò nella **confusione** e nella **diffusione** i due ingredienti di un buon cifrario. La confusione rende complicato il legame tra chiave e cifrato e si ottiene con la **sostituzione**. La diffusione sparge l'effetto di ogni lettera su molte posizioni e si ottiene con la **trasposizione**. DES alterna per 16 round S-box (sostituzioni) e permutazioni.

Il cifrario in PHP

La funzione ripulisce testo e chiave (solo A-Z, maiuscole), riempie con X, calcola l'ordine di lettura delle colonne e poi concatena le colonne. Due dettagli: la lettera in riga `r` e colonna `col` sta nella posizione `r * n + col` del testo scritto per righe; l'ordine delle colonne si ottiene ordinando le posizioni per lettera della chiave, e a parità di lettera per posizione (`usort` con l'operatore `<=>`). Per ZEBRA il risultato è [4, 2, 1, 3, 0].

trasposizione.php

```php
<?php
function ordineColonne(string $chiave): array {
    $lettere = str_split($chiave);
    $posizioni = range(0, count($lettere) - 1);
    usort($posizioni, fn($a, $b) => [$lettere[$a], $a] <=> [$lettere[$b], $b]);
    return $posizioni;
}

function trasposizione(string $testo, string $chiave): string {
    $chiave = strtoupper(preg_replace('/[^A-Za-z]/', '', $chiave));
    $testo = strtoupper(preg_replace('/[^A-Za-z]/', '', $testo));
    $n = strlen($chiave);
    while (strlen($testo) % $n !== 0) {
        $testo .= 'X';
    }
    $righe = intdiv(strlen($testo), $n);
    $cifrato = '';
    foreach (ordineColonne($chiave) as $col) {
        for ($r = 0; $r < $righe; $r++) {
            $cifrato .= $testo[$r * $n + $col];
        }
    }
    return $cifrato;
}
```

Come si rompe: anagrammi e lunghezze

Le frequenze non servono. Si prova una **larghezza** (un divisore della lunghezza del cifrato) e si riordinano le colonne finché le righe formano parole: è un anagramma. Per un cifrato di 20 lettere le larghezze possibili sono i divisori 2, 4, 5 e 10.

Larghezza provata	Altezza delle colonne	Quante permutazioni delle colonne
2	10	2
4	5	24
5	4	120
10	2	3.628.800

Con larghezza 5, per esempio, il cifrato si scrive come cinque colonne verticali e si prova a combinarle in modo che le righe abbiano senso. Le possibili permutazioni sono 5! = 120: poche per un computer, che le prova tutte e tiene quelle che producono sequenze di lettere plausibili in italiano. Per chiavi lunghe il numero cresce molto, ma l'attacco si può fare per gradi: si accostano due colonne alla volta e si guardano le coppie di lettere risultanti.

DA RICORDARE

- La trasposizione cambia l'**ordine** delle lettere, non le lettere: le frequenze restano.
- A colonne: griglia scritta per righe, lettura per colonne nell'ordine alfabetico della chiave, riempimento con **X**.
- Sostituzione (**confusione**) più trasposizione (**diffusione**) è l'idea dei cifrari moderni.

Autoverifica

1. Cifra «RIUNIONE ALLE NOVE» con trasposizione a colonne e chiave SOLE. Quante X servono per completare la griglia?
2. Con la chiave BANANA, in quale ordine si leggono le colonne? Come si risolve la parità delle lettere uguali?
3. Un testo cifrato ha un indice di coincidenza vicino a 0,074 e i picchi di frequenza cadono su E, A, I, O. Che tipo di cifrario è probabile? E se i picchi cadessero su altre lettere?
4. Hai un cifrato di 24 lettere ottenuto con una trasposizione a colonne. Quali larghezze della griglia sono possibili?
5. Perché per decifrare conta la lunghezza del cifrato? Descrivi i passi per ricostruire la griglia.
6. Spiega perché sostituzione e trasposizione si completano. A quale principio di Shannon corrisponde ciascuna?

Per esercitarsi

- Slide del modulo 4
- Simulatore della trasposizione a colonne — con lo strumento per riordinare le colonne
- Bigliettini della trasposizione — da ritagliare, con griglia vuota
- Lab PHP 04: trasposizione a colonne

CAPITOLO 5

Cifrari moderni: bit e XOR

Dai caratteri ai bit, dalla porta XOR ai cifrari a blocchi: le idee che portano al DES.

Dai caratteri ai bit

Un cifrario moderno non sposta lettere: trasforma **sequenze di bit**. Ogni dato (testo, immagine, file) è una sequenza di byte; se il cifrario lavora sui bit, funziona su qualunque dato e non solo sulle lettere dell'alfabeto.

Il testo «Ciao» passa da caratteri a codici numerici, poi a byte (che si scrivono in esadecimale) e infine a bit:

Carattere	C	i	a	o
Codice (decimale)	67	105	97	111
Byte (esadecimale)	43	69	61	6F
Bit	01000011	01101001	01100001	01101111

ASCII e UTF-8

ASCII assegna un numero a ogni carattere base (A = 65, a = 97, cifra 0 = 48, spazio = 32). **UTF-8** lo estende: i caratteri comuni usano 1 byte, gli altri da 2 a 4. Una lettera accentata o il simbolo dell'euro occupano più byte, il che spiega perché nel codice PHP dei capitoli precedenti le lettere accentate non vengono cifrate: ciascun byte è fuori dall'intervallo A-Z.

Carattere	Byte in UTF-8 (esadecimale)	Numero di byte
A	41	1
a	61	1
è	C3A8	2
€	E282AC	3

Esadecimale: i bit in breve

Leggere 64 bit è faticoso. Si raggruppano a **4 bit** e ogni gruppo diventa una cifra: 0-9 e poi A-F. Un byte sono quindi 2 cifre. Il DES scriverà il blocco di 64 bit con 16 cifre esadecimali, per esempio 0123456789ABCDEF.

Cifra	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Decimale	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Bit	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

XOR: l'operazione dei cifrari

XOR (⊕, «o esclusivo») dà **1 se i due bit sono diversi**, 0 se sono uguali.

A	K	A ⊕ K
0	0	0
0	1	1
1	0	1
1	1	0

Su due sequenze di bit si opera bit per bit. Dove la chiave ha un 1 il bit del messaggio viene invertito; dove ha uno 0 resta com'è: la chiave funziona da interruttore.

	bit
A	01101001
K	10110100
A ⊕ K	11011101

Le proprietà

- A ⊕ 0 = A
- A ⊕ A = 0
- A ⊕ K = K ⊕ A
- (A ⊕ K) ⊕ K = A ⊕ (K ⊕ K) = A ⊕ 0 = **A**

> **CONSEGUENZA**
>
> Per cifrare e per decifrare si fa la **stessa operazione** con la stessa chiave. Non esiste un «algoritmo inverso». Verifica sul byte di prima: (A ⊕ K) ⊕ K = 01101001 = A.

Il cifrario XOR sul testo

Si trasforma il messaggio in byte e si fa lo XOR con la chiave, **ripetendola** se è più corta del messaggio.

Esempio svolto: messaggio «CIAO» con chiave «LUNA»

Carattere	C	I	A	O
Messaggio (bit)	01000011	01001001	01000001	01001111
Chiave «LUNA» (bit)	01001100	01010101	01001110	01000001
XOR (bit)	00001111	00011100	00001111	00001110
XOR (esadecimale)	0F	1C	0F	0E

Il cifrato è `0F1C0F0E` (esadecimale). Per decifrare basta rifare lo XOR con la stessa chiave: `0F1C0F0E` ⊕ `4C554E41` = `4349414F`, cioè «CIAO». I byte del cifrato sono spesso caratteri non stampabili, per questo si mostra l'esadecimale.

One-time pad: la cifratura perfetta

> **LE TRE REGOLE**
> Se la chiave è **casuale**, **lunga quanto il messaggio** e **usata una volta sola**, il cifrario XOR è il one-time pad. Claude Shannon dimostrò nel 1949 che in queste condizioni il cifrato non rivela nulla sul messaggio (segretezza perfetta).

Se la chiave si riusa, crolla tutto

Con la stessa chiave su due messaggi, lo XOR dei cifrati **cancella la chiave**: C1 ⊕ C2 = (M1 ⊕ K) ⊕ (M2 ⊕ K) = M1 ⊕ M2. Esempio con i messaggi «SI» e «NO» e la chiave «q7»:

	Primo byte	Secondo byte
M1 «SI»	01010011	01001001
M2 «NO»	01001110	01001111
M1 ⊕ M2	00011101	00000110
C1 = M1 ⊕ K	00100010	01111110
C2 = M2 ⊕ K	00111111	01111000
C1 ⊕ C2	00011101	00000110

Le ultime righe delle due metà coincidono: chi intercetta i due cifrati ottiene M1 ⊕ M2 senza conoscere la chiave, e da lì, con un po' di ingegno o di testo noto, può risalire ai messaggi. Nel progetto Venona i crittoanalisti statunitensi lessero parti di messaggi sovietici proprio perché alcune pagine di one-time pad erano state riusate: «una volta sola» non è una raccomandazione.

Chiave corta e ripetuta: debole

Se la chiave è più corta del messaggio, si ripete; chi conosce (o indovina) l'inizio del testo la legge direttamente. Se Eve sospetta che il messaggio cominci con «Gent», fa lo XOR tra i primi byte del cifrato e quelli dell'ipotesi:

	G	e	n	t
Cifrato (esadecimale)	0B	30	20	35
Ipotesi «Gent» (esadecimale)	47	65	6E	74
XOR = chiave	4C	55	4E	41
Come testo	L	U	N	A

Compare la chiave `LUNA`, che ripetuta sul resto del cifrato decifra tutto il messaggio. Un attacco di questo tipo si chiama *a testo noto* (known plaintext).

Confusione e diffusione

Nel 1949 Claude Shannon, in «Communication Theory of Secrecy Systems», indicò due proprietà che un buon cifrario deve avere. Sono ancora il criterio con cui si progettano i cifrari a blocchi.

Proprietà	Che cosa richiede	Come si ottiene
Confusione	Il legame tra chiave e cifrato deve essere complicato: ogni bit del cifrato dipende dalla chiave in modo non lineare	Sostituzione (S-box)
Diffusione	Ogni bit del messaggio deve influenzare molti bit del cifrato, nascondendo le statistiche	Permutazione e più round

Cesare e il solo XOR hanno confusione e diffusione molto basse: un bit cambiato in ingresso cambia un solo bit in uscita.

Effetto valanga

Un buon cifrario fa sì che cambiando **un solo bit** del messaggio cambi circa **metà** dei bit del cifrato. Con il DES (chiave 133457799BBCDFF1) si cifrano due blocchi che differiscono per un solo bit: 0123456789ABCDEF e 0123456789ABCDEE.

Dopo il round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bit diversi su 64	1	6	22	32	30	30	29	29	31	31	37	36	33	39	37	37

Dopo pochi round i due blocchi differiscono già in circa metà dei bit; alla fine i due cifrati (`85E813540F0AB405` e `28378E295BE22A84`) differiscono per `37` bit su 64. Con un cifrario XOR la differenza sarebbe stata di 1 bit.

Cifrari a flusso e a blocchi

	A flusso	A blocchi
Idea	Genera una sequenza pseudocasuale di bit dalla chiave e la combina con il messaggio con XOR, bit per bit	Divide il messaggio in blocchi di lunghezza fissa; ogni blocco passa per più round di trasformazione
Adatto a	Dati continui (comunicazioni in tempo reale)	Dati di qualsiasi lunghezza, con una modalità operativa (ECB, CBC…)
Esempi	ChaCha20, RC4 (oggi sconsigliato)	DES (64 bit), AES (128 bit)

Rete di sostituzione-permutazione

Un cifrario a blocchi ripete più volte un round fatto di tre operazioni: XOR con una sottochiave del round, **S-box** (sostituzione: confusione), **P-box** (permutazione: diffusione). Ripetendo i round, ogni bit del blocco finisce per dipendere da ogni bit della chiave e del messaggio.

L'idea della rete di Feistel

Il DES non è una rete SP pura: usa la struttura di **Feistel**. Si divide il blocco in due metà, L (sinistra) e R (destra), e in ogni round: R passa invariata e diventa la nuova L; la funzione F(R, K) calcolata con la sottochiave viene combinata con XOR alla vecchia L, e il risultato diventa la nuova R.

> **IL VANTAGGIO**
>
> La funzione F **non deve essere invertibile**: per decifrare si ripete la stessa struttura con le sottochiavi in ordine inverso. Nel capitolo 6 lo si vede passo per passo.

DA RICORDARE

- Il computer cifra **bit**: testo → byte (ASCII/UTF-8) → esadecimale.
- XOR: **A ⊕ K ⊕ K = A**. Con chiave casuale, lunga come il messaggio e usata una volta è perfetto; altrimenti è debole.
- Shannon: **confusione** (S-box) e **diffusione** (permutazioni, round, effetto valanga).
- Cifrari a flusso e a blocchi; reti SP e reti di Feistel.

Autoverifica

1. Scrivi in binario e in esadecimale la parola «Re» (codici ASCII 82 e 101).
2. Calcola 10110010 ⊕ 01101100 e poi rifai lo XOR con 01101100: che cosa ottieni, e perché?
3. Elenca le tre condizioni del one-time pad. Che cosa succede se la chiave viene usata due volte?
4. Perché un cifrario XOR con chiave corta e ripetuta è debole se si conosce l'inizio del messaggio?
5. Spiega confusione e diffusione con un esempio per ciascuna. Perché Cesare non ha né l'una né l'altra?
6. Descrivi un round di una rete di Feistel con le formule L′ = … e R′ = …; perché F non deve essere invertibile?

Per esercitarsi

- Slide del modulo 5
- Simulazione: XOR bit a bit — anche da smartphone

CAPITOLO 6

DES

Il cifrario a blocchi che ha dominato per vent'anni: struttura di Feistel, S-DES a mano, DES reale, modalità operative e successori.

Da Lucifer al DES

Quando	Che cosa è successo
Anni '70	IBM, con Horst Feistel, sviluppa il cifrario Lucifer (chiave a 128 bit nelle versioni descritte).
1973-1974	Il National Bureau of Standards (NBS, oggi NIST) chiede proposte per uno standard di cifratura civile. IBM propone una variante di Lucifer.
1977	L'NBS pubblica il DES come standard federale (FIPS 46): blocchi di 64 bit, chiave di 56 bit.
1999-2005	Si affianca la versione Triple DES (FIPS 46-3); nel 2001 nasce AES (FIPS 197) e nel 2005 il DES viene ritirato come standard.

Per la prima volta un algoritmo di cifratura **pubblico e standardizzato** poteva essere studiato da tutti. Ha ispirato quasi tutta la ricerca successiva sui cifrari a blocchi.

Le polemiche: chiave corta e S-box

- **56 bit, non 128**: Lucifer usava chiavi più lunghe. Gli studiosi dell'epoca criticarono la scelta: uno spazio di 2^{56} chiavi sarebbe stato attaccabile con hardware dedicato.
- **Il ruolo della NSA**: l'agenzia collaborò alla revisione e si sospettò un «passaggio segreto». Un'indagine del Senato statunitense (1978) non trovò manomissioni dell'algoritmo.
- **Il sospetto che rientrò**: nel 1990 Biham e Shamir scoprirono la crittoanalisi differenziale e si vide che le S-box del DES le resistevano bene. Coppersmith (IBM) raccontò nel 1994 che il team la conosceva già dal 1974 e aveva progettato le S-box di conseguenza.

Il DES in una figura

Blocco in chiaro	IP	16 round (Feistel)	Scambio L/R	IP^{-1}	Blocco cifrato
64 bit = 16 cifre esadecimali	permutazione iniziale	ognuno con la propria sottochiave	dopo il round 16	permutazione finale	64 bit

- **Blocco**: 64 bit in ingresso e in uscita.
- **Chiave**: 64 bit, ma 8 sono di parità: solo **56** contano. Da essa si ricavano 16 sottochiavi da 48 bit.
- **Round**: 16, ognuno con la propria sottochiave.

L'esempio che useremo è il vettore di prova ufficiale: chiave `133457799BBCDFF1`, blocco `0123456789ABCDEF`, cifrato atteso `85E813540F0AB405`.

La rete di Feistel

Il blocco si divide in due metà, L (sinistra) e R (destra). In ogni round, con K_i sottochiave del round:

$$L_i = R_{i-1} \qquad R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Metà del blocco (R) passa invariata e diventa la nuova L; l'altra metà viene mescolata con lo XOR con il risultato di F, la «funzione complicata». Al round successivo le parti si invertono.

Perché si può decifrare

Un round all'andata	Come tornare indietro
$L_1 = R_0$	$R_0 = L_1$
$R_1 = L_0 \oplus F(R_0, K_1)$	$L_0 = R_1 \oplus F(L_1, K_1)$

Il passaggio funziona perché $R_1 \oplus F(R_0, K_1) = L_0 \oplus F \oplus F = L_0$, grazie alla proprietà $A \oplus K \oplus K = A$ del capitolo 5.

> **CONSEGUENZA**
>
> Per decifrare si applica la stessa struttura, partendo dall'ultimo round, con **K_{16} per primo e K_1 per ultimo**. F non deve essere invertibile: basta calcolarla, non annullarla.

S-DES: il DES in miniatura

Il DES a 16 round è troppo lungo da seguire a mano. **S-DES** (Simplified DES, versione didattica di Stallings) ha la stessa struttura in piccolo.

	DES	S-DES
Blocco	64 bit	8 bit
Chiave	56 bit (64 con parità)	10 bit
Round	16	2
Sottochiavi	16 × 48 bit	2 × 8 bit
S-box	8 (6 → 4 bit)	2 (4 → 2 bit)

> **ATTENZIONE**
>
> S-DES è solo un esercizio: $2^{10} = 1024$ chiavi si provano tutte in un istante. Serve a capire, non a proteggere.

Le tabelle e come si leggono

Una **tabella di permutazione** elenca, per ogni posizione di *uscita*, da quale bit dell'*ingresso* prendere (i bit si numerano da 1). Esempio con P4 = 2 4 3 1 su 1011: l'uscita prende il bit 2 (0), poi il 4 (1), poi il 3 (1), poi il 1 (1) e vale 0111. P8 scarta 2 dei 10 bit; E/P ne ripete alcuni per passare da 4 a 8.

Tabella	Valori
P10	3 5 2 7 4 10 1 9 8 6
P8	6 3 7 4 8 5 10 9
IP	2 6 3 1 4 8 5 7
IP^{-1}	4 1 3 5 7 2 8 6
E/P	4 1 2 3 2 3 4 1
P4	2 4 3 1

Leggere una S-box

Una S-box trasforma pochi bit in meno bit usando una tabella. Dei 4 bit in ingresso $b_1b_2b_3b_4$, i bit esterni (b_1 e b_4) scelgono la **riga**, quelli interni (b_2 e b_3) la **colonna**. Il valore trovato (0-3) si riscrive su 2 bit.

S0	col. 0	col. 1	col. 2	col. 3
riga 0	1	0	3	2
riga 1	3	2	1	0
riga 2	0	2	1	3
riga 3	3	1	3	2

S1	col. 0	col. 1	col. 2	col. 3
riga 0	0	1	2	3
riga 1	2	0	1	3
riga 2	3	0	1	0
riga 3	2	1	0	3

Esempio svolto: S-DES con chiave 1010000010 e blocco 01110010

Le due sottochiavi

1. **P10** sulla chiave: `1010000010` → `1000001100`. Si divide in due metà da 5 bit: C_0 = `10000`, D_0 = `01100`.
2. **Scorrimento a sinistra di 1** (LS-1) di ciascuna metà: C = `00001`, D = `11000`, in tutto `0000111000`.
3. **P8** sceglie 8 dei 10 bit: **K1 = 10100100**.
4. Sulle metà già scorse si applica **LS-2** (scorrimento di altri 2 posti): `0010000011`.
5. **P8** di nuovo: **K2 = 01000011**.

Cifrare il blocco

Il blocco `01110010` passa per **IP** e diventa `10101001`; si divide in L = `1010` e R = `1001`. Poi due round identici, con K1 nel primo e K2 nel secondo. Dentro un round, la funzione F lavora così:

1. E/P espande R da 4 a 8 bit;
2. XOR con la sottochiave;

3. i 4 bit di sinistra vanno in S0, i 4 di destra in S1 (2 bit ciascuna);
4. i 4 bit in uscita dalle S-box passano per P4;
5. il risultato si combina con XOR con L, e le metà si scambiano.

Passo	Round 1 (K1 = 10100100)	Round 2 (K2 = 01000011)
Ingresso (L, R)	1010 1001	1001 1101
E/P di R	11000011	11101011
XOR con la sottochiave	01100111	10101000
Ingresso S0 | S1	0110 | 0111	1010 | 1000
S0	riga 0, colonna 3 → 2 = 10	riga 2, colonna 1 → 2 = 10
S1	riga 1, colonna 3 → 3 = 11	riga 2, colonna 0 → 3 = 11
Uscite S0 S1	1011	1011
P4 = F(R, K)	0111	0111
L ⊕ F	1101	1110
Uscita del round	11011001	11101101

Tra i due round le metà si scambiano (SW): `11011001` → `10011101`. Dopo il secondo round (senza scambio) si ottiene `11101101`. La permutazione finale $\mathbf{IP^{-1}}$ dà il cifrato:

01110010 → 01110111

Decifrare

Per decifrare si ripete **tutto** l'algoritmo, ma nel primo round si usa K2 e nel secondo K1.

	Cifratura	Decifratura
Ingresso	01110010	01110111
Round 1 usa	K1 = 10100100	K2 = 01000011
Round 2 usa	K2 = 01000011	K1 = 10100100
Uscita	01110111	01110010

Verifica: il cifrato `01110111` decifrato con la stessa chiave ritorna al blocco di partenza `01110010`.

Il DES reale

IP e IP^{-1}: le permutazioni iniziale e finale

Le permutazioni del DES funzionano come quelle di S-DES, con 64 posizioni. Le tabelle si leggono per righe: la prima riga di IP (58, 50, 42, …) dice che il bit 1 dell'uscita è il bit 58 dell'ingresso.

IP (64 → 64)							
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

IP^{-1} (64 → 64)							
40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

Il blocco `0123456789ABCDEF` in binario è `00000001 00100011 01000101 01100111 10001001 10101011 11001101 11101111`. Dopo IP diventa `11001100 00000000 11001100 11111111 11110000 10101010 11110000 10101010`, da cui L_0 = `CC00CCFF` e R_0 = `F0AAF0AA` (le due metà da 32 bit, in esadecimale). IP^{-1} è la permutazione inversa: annulla IP.

Le sottochiavi: PC-1, rotazioni, PC-2

1. **PC-1** (da 64 a 56 bit) scarta gli 8 bit di parità e rimescola i 56 rimasti, che si dividono in due metà da 28 bit: C_0 e D_0. Per la chiave `133457799BBCDFF1`: C_0 = `1111000 0110011 0010101 0101111`, D_0 = `0101010 1011001 1001111 0001111`.
2. In ogni round C e D **ruotano a sinistra** di 1 o 2 posti, secondo una tabella fissa. Le rotazioni sono cumulative.
3. **PC-2** sceglie 48 dei 56 bit di C_iD_i: ecco la sottochiave K_i.

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Rotazioni	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

PC-1 (64 → 56)						
57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

PC-2 (56 → 48)					
14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

Dopo il round 1 (rotazione di 1): C_1 = `1110000 1100110 0101010 1011111`, D_1 = `1010101 0110011 0011110 0011110`. Con PC-2 si ottiene K_1 = `1B02EFFC7072` (48 bit = 12 cifre esadecimali). Tutte le sottochiavi dell'esempio:

Sottochiave	Valore (hex)	Sottochiave	Valore (hex)
K1	1B02EFFC7072	K9	E0DBEBEDE781
K2	79AED9DBC9E5	K10	B1F347BA464F
K3	55FC8A42CF99	K11	215FD3DED386
K4	72ADD6DB351D	K12	7571F59467E9
K5	7CEC07EB53A8	K13	97C5D1FABA41
K6	63A53E507B2F	K14	5F43B7F2E73A
K7	EC84B7F618BC	K15	BF918D3D3F0A
K8	F78A3AC13BFB	K16	CB3D8B0E17F5

La funzione F del DES

La funzione F prende R (32 bit) e la sottochiave K (48 bit):

1. **E** (espansione) porta R da 32 a 48 bit ripetendo i bit ai bordi di ogni gruppo da 4;
2. XOR con la sottochiave K;
3. le **8 S-box**: ogni gruppo da 6 bit entra in una S-box e ne esce un gruppo da 4 bit (bit esterni = riga, bit interni = colonna);
4. **P** permuta i 32 bit risultanti.

E (32 → 48)					
32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

P (32 → 32)			
16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

Le 8 S-box sono l'unica parte **non lineare** (confusione): senza di esse il DES sarebbe una funzione lineare, fatta di XOR e permutazioni, e si romperebbe risolvendo equazioni. L'espansione E e la permutazione P distribuiscono l'effetto di ogni bit su più S-box del round successivo (diffusione). Come esempio ecco la prima S-box; le altre sette sono nello standard FIPS 46-3.

S1	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
riga 0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
riga 1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
riga 2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
riga 3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

Esempio svolto: il primo round del DES

Valori con il vettore ufficiale: R_0 = `F0AAF0AA` (in binario `11110000 10101010 11110000 10101010`), K_1 = `1B02EFFC7072`.

1. **E(R_0)** (48 bit): `011110 100001 010101 010101 011110 100001 010101 010101`
2. **XOR con K_1**: `011000 010001 011110 111010 100001 100110 010100 100111`
3. Gruppi da 6 bit nelle 8 S-box (riga = primo e ultimo bit, colonna = i 4 bit centrali):

S-box	Ingresso (6 bit)	Riga	Colonna	Valore	Uscita (4 bit)
S1	011000	0	12	5	0101
S2	010001	1	8	12	1100
S3	011110	0	15	8	1000
S4	111010	2	13	2	0010
S5	100001	3	0	11	1011
S6	100110	2	3	5	0101
S7	010100	0	10	9	1001
S8	100111	3	3	7	0111

1. Le 8 uscite affiancate: `0101 1100 1000 0010 1011 0101 1001 0111` (32 bit).
2. **P**: `0010 0011 0100 1010 1010 1001 1011 1011` = `234AA9BB` = F(R_0, K_1).
3. **$R_1 = L_0 \oplus F$** = `CC00CCFF` ⊕ `234AA9BB` = **`EF4A6544`**; **$L_1 = R_0$** = **`F0AAF0AA`**.

I 16 round e il risultato finale

Si ripete lo stesso round 16 volte con $K_1 \ldots K_{16}$. La tabella riporta le due metà dopo ciascun round (esadecimale).

Round	L	R	Round	L	R
1	F0AAF0AA	EF4A6544	9	D5694B90	247CC67A
2	EF4A6544	CC017709	10	247CC67A	B7D5D7B2
3	CC017709	A25C0BF4	11	B7D5D7B2	C5783C78
4	A25C0BF4	77220045	12	C5783C78	75BD1858
5	77220045	8A4FA637	13	75BD1858	18C3155A
6	8A4FA637	E967CD69	14	18C3155A	C28C960D
7	E967CD69	064ABA10	15	C28C960D	43423234
8	064ABA10	D5694B90	16	43423234	0A4CD995

Dopo il round 16 le metà si scambiano: $R_{16}L_{16}$ = `0A4CD99543423234`. Infine IP^{-1} produce il blocco cifrato.

> **VETTORE DI PROVA**
> Chiave **133457799BBCDFF1** · testo **0123456789ABCDEF** → **85E813540F0AB405**. È il vettore di prova ufficiale con cui si verifica ogni implementazione del DES.

Modalità operative

Il DES cifra 8 byte per volta. Per messaggi più lunghi serve una **modalità operativa**.

ECB: blocchi uguali, cifrati uguali

In **ECB** (Electronic Codebook) ogni blocco si cifra da solo con la stessa chiave: blocchi uguali producono cifrati uguali, e la struttura dei dati resta visibile (il caso classico è un'immagine cifrata in ECB di cui si riconosce ancora la sagoma).

CBC e il vettore di inizializzazione

In **CBC** (Cipher Block Chaining) ogni blocco in chiaro viene combinato con XOR con il cifrato del blocco precedente prima di essere cifrato; per il primo blocco si usa un **vettore di inizializzazione** (IV). L'IV è casuale, non segreto, e va cambiato a ogni messaggio.

Esempio svolto: tre blocchi, due uguali

Si cifrano con la chiave `133457799BBCDFF1` tre blocchi da 8 byte (due uguali: «ATTACCO!» ripetuto, poi «LUNA!!!!»). IV per CBC: `A1B2C3D4E5F60718`.

Blocco	In chiaro (hex)	ECB	CBC
1	4154544143434F21	**9238ADD846EDE8A4**	F1F1977DBEEEF655
2	4154544143434F21	**9238ADD846EDE8A4**	7C79E7E847712582
3	4C554E4121212121	12B99EF44431E793	0C0CA217FE30A319

In ECB i blocchi 1 e 2 hanno lo stesso cifrato; in CBC no, perché ogni blocco è combinato con il precedente.

Il problema dei 56 bit

Le chiavi possibili sono 2^{56} = 72.057.594.037.927.936, circa $7{,}2 \times 10^{16}$. Il tempo per provarle tutte dipende dalla velocità:

Velocità di ricerca	Tempo per provarle tutte
1 milione di chiavi al secondo	2285 anni
1 miliardo di chiavi al secondo	2,3 anni
92 miliardi di chiavi al secondo (Deep Crack, dato EFF)	9,1 giorni

In media si trova la chiave dopo aver provato metà dello spazio: la metà dei tempi indicati. Con hardware specializzato e molti chip in parallelo la ricerca diventa realistica.

Deep Crack, 1998

Quando	Chi	Risultato
1997	Progetto DESCHALL (distribuito)	Prima «DES Challenge» di RSA risolta in 96 giorni.
Gen-feb 1998	distributed.net	DES Challenge II-1 risolta in 39 giorni.
Luglio 1998	EFF, macchina «Deep Crack»	DES Challenge II-2 risolta in 56 ore. Costo di costruzione sotto i 250.000 dollari.
Gen 1999	EFF e distributed.net	DES Challenge III risolta in circa 22 ore.

Il DES con una sola chiave a 56 bit non è più sicuro: la dimostrazione pubblica che un attacco a forza bruta era alla portata di organizzazioni con pochi mezzi.

I successori: Triple DES e AES

Triple DES (3DES)

Il Triple DES applica il DES tre volte nello schema EDE: cifra con K1, *decifra* con K2, cifra con K3. Perché «decifra» al centro? Con K1 = K2 = K3 le tre operazioni si semplificano e si ottiene il DES singolo: compatibilità con i sistemi vecchi.

Chiavi	Blocco 0123456789ABCDEF diventa
K1 = K2 = K3 = 133457799BBCDFF1	85E813540F0AB405 (uguale al DES singolo)
K1 = 133457799BBCDFF1, K2 = 0E329232EA6D0D73, K3 = 8787878787878787	004ECBE74DE738A9

La chiave ha 168 bit (3 × 56), ma l'attacco «meet-in-the-middle» riduce la sicurezza effettiva a circa **112 bit**. Resta lento e con blocchi di 64 bit: oggi è sconsigliato.

AES

Nel 1997 il NIST lancia una gara pubblica; vince **Rijndael** (Daemen e Rijmen, Belgio). AES diventa standard nel 2001 (FIPS 197).

Parametro	AES
Blocco	128 bit
Chiave	128, 192 o 256 bit
Round	10, 12 o 14
Struttura	rete di sostituzione-permutazione (non Feistel): tutto il blocco è trasformato a ogni round

AES-128 e AES-256 sono oggi lo standard di fatto (Wi-Fi WPA2, HTTPS, dischi cifrati). Le idee viste nel DES (confusione, diffusione, round, sottochiavi) si ritrovano tutte.

Il DES in PHP con OpenSSL

Per usare un cifrario a blocchi in PHP non si riscrive l'algoritmo: si usa la funzione `openssl_encrypt()`. Esempio con 3DES in modalità CBC. `random_bytes()` produce byte casuali crittograficamente sicuri; l'IV ha la lunghezza del blocco (8 byte) e viaggia in chiaro davanti al cifrato; OpenSSL aggiunge da solo il padding.

3des-cbc.php

```
<?php
$chiave = random_bytes(24);          // 3DES: tre chiavi DES da 8 byte
$iv     = random_bytes(8);           // blocco DES = 8 byte, quindi IV di 8 byte
$testo  = 'Messaggio segreto';

$cifrato = openssl_encrypt($testo, 'des-ede3-cbc', $chiave, OPENSSL_RAW_DATA, $iv);
$invio   = base64_encode($iv . $cifrato);      // IV in chiaro davanti al cifrato

$dati  = base64_decode($invio);
$iv2   = substr($dati, 0, 8);
$chiaro = openssl_decrypt(substr($dati, 8), 'des-ede3-cbc', $chiave, OPENSSL_RAW_DATA, $iv2);
echo $chiaro;
```

E il DES singolo?

des-singolo.php

```php
<?php
$m = hex2bin('0123456789ABCDEF');
$k = hex2bin('133457799BBCDFF1');

// DES singolo: con OpenSSL 3 e' nel "provider legacy", di solito disattivato
var_dump(openssl_encrypt($m, 'DES-ECB', $k, OPENSSL_RAW_DATA));   // bool(false)

// Trucco: 3DES con K1 = K2 = K3 coincide con il DES singolo
$c = openssl_encrypt($m, 'des-ede-ecb', $k . $k, OPENSSL_RAW_DATA | OPENSSL_ZERO_PADDING);
echo strtoupper(bin2hex($c));        // 85E813540F0AB405
```

COMPORTAMENTO REALE

Con PHP 8.3 e OpenSSL 3 la chiamata `openssl_encrypt(…, 'DES-ECB', …)` restituisce `false` (errore «unsupported»): in OpenSSL 3 il DES singolo sta nel *provider legacy*, spesso non attivo. 3DES e AES funzionano. Con K1 = K2 = K3 il 3DES dà lo stesso risultato del vettore ufficiale 85E813540F0AB405.

DA RICORDARE

- Feistel: **L′ = R, R′ = L ⊕ F(R, K)**; per decifrare stessa struttura, sottochiavi invertite.
- DES: blocco 64 bit, chiave 56 bit, 16 round; F = E, XOR con la sottochiave, 8 S-box, P.
- ECB lascia vedere la struttura dei dati; CBC la nasconde con IV e concatenamento.
- 56 bit si forzano (Deep Crack, 1998): 3DES e poi AES.

Autoverifica

1. Scrivi le formule di un round di Feistel e spiega perché per decifrare basta invertire l'ordine delle sottochiavi.
2. Calcola con S-DES le sottochiavi K1 e K2 per la chiave 1100011110 (usa le tabelle P10, P8 e gli scorrimenti).
3. In un round DES, come si passa da 32 a 48 bit? E da 48 a 32? Quale delle due operazioni è non lineare?
4. Che differenza c'è tra ECB e CBC? Perché l'IV non deve essere segreto ma deve cambiare a ogni messaggio?
5. Quante chiavi ha il DES? Perché Deep Crack nel 1998 rese insicura la chiave da 56 bit?
6. Perché il 3DES usa lo schema cifra-decifra-cifra? Quale sicurezza effettiva offre?

Per esercitarsi

- Slide del modulo 6
- Simulatore S-DES passo per passo — con input libero
- Simulatore DES: 16 round
- Gioco: S-DES a mano — chiave e blocco casuali, verifica a ogni passo
- Scheda S-DES a mano — da stampare
- Lab PHP 07: DES e OpenSSL — openssl_encrypt, ECB contro CBC

CAPITOLO 7

La matematica di RSA

Orologi, resti, numeri primi e potenze: gli attrezzi con cui si costruisce il cifrario a chiave pubblica.

Che cosa ci serve per RSA

RSA usa pochi ingredienti, ma bisogna padroneggiarli uno per uno. Questo capitolo li costruisce in ordine:

Ingrediente	A che cosa serve
Aritmetica modulare	I numeri «girano», come su un orologio: tutti i calcoli di RSA sono resti
MCD e numeri coprimi	Verificare che e sia utilizzabile (Euclide)
Inverso modulare	Calcolare la chiave privata d (Euclide esteso)
Funzione φ di Eulero	Il numero «segreto» φ(n) per n = p · q e il perché RSA funziona
Potenza modulare	Cifrare e decifrare con numeri enormi (square-and-multiply)
Funzione con botola	Facile in un verso, difficile nell'altro: la sicurezza

L'orologio: i numeri girano

Sono le 9 e passano 5 ore: non sono le 14, sono le 2. Su un orologio da 12 ore si calcola (9 + 5) mod 12 = 2: dopo il 12 si riparte da 0. Lo stesso accade con le lettere nel cifrario di Cesare, dove l'orologio ha 26 ore.

Resti e congruenze

- **a mod n** è il resto della divisione di a per n, sempre compreso tra 0 e n − 1.
- **a ≡ b (mod n)** («a è congruo a b modulo n») significa che a e b hanno lo stesso resto: stanno sulla stessa lancetta.

a	n	a mod n	Spiegazione
17	5	2	17 = 3 · 5 + 2
100	7	2	100 = 14 · 7 + 2
−3	26	23	−3 + 26 = 23 (resto sempre non negativo)
3233	61	0	3233 = 53 · 61 + 0: 3233 è multiplo di 61

> **ERRORE TIPICO**
> Le funzioni `%` di PHP e JavaScript restituiscono un resto negativo se il dividendo è negativo (`-3 % 26` vale −3). Il resto matematico va ricavato con `(($a % $n) + $n) % $n`.

Calcolare in modulo senza numeri enormi

Si può **ridurre prima** e calcolare dopo:

$$(a \cdot b) \bmod n = ((a \bmod n) \cdot (b \bmod n)) \bmod n$$

La regola vale per somma, sottrazione e prodotto. La divisione no: per dividere serve l'inverso, che vediamo tra poco.

Esempio svolto: $(123 \cdot 456) \bmod 7$

Strada lunga: $123 \cdot 456 = 56.088$, poi $56.088 \bmod 7 = 4$. Strada breve: $123 \bmod 7 = 4$, $456 \bmod 7 = 1$, quindi $(4 \cdot 1) \bmod 7 = 4$. Il risultato è lo stesso, ma i numeri non crescono mai. Questa è la proprietà che permetterà di calcolare potenze enormi.

L'inverso moltiplicativo e la tabella del prodotto

L'**inverso** di a modulo n è il numero x tale che **$a \cdot x \equiv 1 \pmod n$**. È come «dividere per a»: in RSA serve per ricavare la chiave privata. Per esempio $3 \cdot 5 = 15 \equiv 1 \pmod 7$, quindi l'inverso di 3 modulo 7 è 5.

> **QUANDO ESISTE**
>
> L'inverso di a modulo n esiste se e solo se **MCD(a, n) = 1**, cioè se a e n sono coprimi.

Per vedere chi ha un inverso si cerca il numero 1 in ogni riga della tabella del prodotto. Modulo 7 (primo) ogni riga contiene un 1; modulo 8 le righe dei numeri pari non ne contengono.

×	1	2	3	4	5	6
1	**1**	2	3	4	5	6
2	2	4	6	**1**	3	5
3	3	6	2	5	**1**	4
4	4	**1**	5	2	6	3
5	5	3	**1**	6	4	2
6	6	5	4	3	2	**1**

Tabella del prodotto modulo 7. Ogni numero ha un inverso: gli 1 sono evidenziati.

×	1	2	3	4	5	6	7
1	**1**	2	3	4	5	6	7
2	2	4	6	0	2	4	6
3	3	6	**1**	4	7	2	5
4	4	0	4	0	4	0	4
5	5	2	7	4	**1**	6	3
6	6	4	2	0	6	4	2
7	7	6	5	4	3	2	**1**

Tabella del prodotto modulo 8. I numeri pari (2, 4, 6) non hanno un 1 nella riga: non hanno inverso, perché non sono coprimi con 8.

Numeri primi e coprimi

- **Primo**: ha solo due divisori, 1 e se stesso. 2, 3, 5, 7, 11, … 53, 61.
- **Coprimi**: due numeri sono coprimi se non hanno fattori in comune, cioè se hanno **MCD = 1**. 8 e 15 sono coprimi anche se nessuno dei due è primo.

Il massimo comun divisore

MCD(a, b) è il più grande numero che divide sia a sia b. Il metodo della scuola scompone in fattori primi e prende quelli comuni: $36 = 2^2 \cdot 3^2$ e $60 = 2^2 \cdot 3 \cdot 5$, quindi MCD = $2^2 \cdot 3 = 12$. Ma scomporre è lento per numeri grandi, e per RSA servono numeri con centinaia di cifre: bisogna calcolare il MCD **senza** fattorizzare.

L'algoritmo di Euclide

> **IDEA DI EUCLIDE**
> MCD(a, b) = MCD(b, a mod b). Un numero che divide a e b divide anche il resto: si rimpicciolisce il problema finché il resto è zero. L'ultimo resto diverso da zero è il MCD.

Esempio svolto: MCD(3120, 17)

Si divide, si prende il resto, si scala di un posto (il divisore diventa dividendo, il resto diventa divisore):

Divisione	a	b	Quoziente	Resto
3120 = 183 · 17 + 9	3120	17	183	9
17 = 1 · 9 + 8	17	9	1	8
9 = 1 · 8 + 1	9	8	1	**1**
8 = 8 · 1 + 0	8	1	8	0

L'ultimo resto diverso da zero è 1: MCD(3120, 17) = `1`. I due numeri sono coprimi.

Euclide esteso e l'inverso

Oltre al MCD, l'algoritmo può fornire due interi **s** e **t** tali che **MCD(a, b) = a·s + b·t** (i coefficienti di Bézout). Si mantengono, accanto ai resti, due colonne s e t con la stessa regola dei resti: ogni riga si ottiene sottraendo, dalla riga di due posizioni prima, il quoziente per la riga precedente. Si parte da (r, s, t) = (a, 1, 0) e (b, 0, 1).

Esempio svolto: i coefficienti di Bézout di 3120 e 17

Passo	Quoziente q	Resto r	s	t
0	–	3120	1	0
1	–	17	0	1
2	183	9	1	−183
3	1	8	−1	184
4	**1**	**1**	**2**	**−367**
5	8	0	−17	3120

Nella penultima riga il resto è 1, con s = 2 e t = −367: `1 = 3120 · 2 + 17 · (−367)`. Si verifica: 6240 − 6239 = 1.

Dall'esteso all'inverso

Da 1 = 3120·s + 17·t, lavorando modulo 3120 il primo termine sparisce e resta 17·t ≡ 1: **t è l'inverso di 17 modulo 3120**. Qui t = −367, che modulo 3120 vale 3120 − 367 = 2753.

Verifica: 17 · 2753 = 46.801 = 15 · 3120 + 1. Quindi `17⁻¹ ≡ 2753 (mod 3120)`. Questo numero sarà la chiave privata d di RSA nel capitolo 8.

La funzione φ di Eulero

φ(n) è il numero di interi tra 1 e n che sono **coprimi** con n. Due regole bastano per RSA:

- φ(p) = p − 1 se p è primo;
- φ(p · q) = (p − 1)(q − 1) se p e q sono primi distinti.

Esempio: φ(15) = φ(3·5) = 2 · 4 = 8, e infatti i numeri tra 1 e 15 coprimi con 15 sono 1, 2, 4, 7, 8, 11, 13, 14: otto numeri. Per i numeri dell'esempio RSA, φ(3233) = φ(61·53) = 60 · 52 = 3120.

> **LA PROPRIETÀ MAGICA**
>
> Se **MCD(a, n) = 1** allora $\mathbf{a^{\varphi(n)} \equiv 1 \pmod n}$ (teorema di Eulero).

Esempio svolto: n = 15, φ(n) = 8

Per a = 2 (coprimo con 15): $2^8 = 256 = 17 \cdot 15 + 1$, quindi $2^8 \equiv 1 \pmod{15}$. Per a = 3 e a = 5, invece, che non sono coprimi con 15 (hanno fattori in comune), la regola non vale: $3^8 \bmod 15 = 6$ e $5^8 \bmod 15 = 10$, non 1.

Potenza modulare: square-and-multiply

Cifrare con RSA significa calcolare $m^e \bmod n$. La strada ingenua è calcolare prima la potenza e poi il resto. Con numeri veri è impraticabile:

Caso	Potenza	Cifre del risultato
65^17 (esempio giocattolo)	65997435908365920509338378906 25	31
m a 2048 bit, e = 65537	(2^2048)^65537	circa 40.404.179

Con chiavi vere il numero intermedio avrebbe oltre 40.404.179 cifre decimali, per ogni singola cifratura. Il risultato finale, invece, ha al massimo 617 cifre. La soluzione è ridurre modulo n a ogni moltiplicazione, così i numeri non superano mai n.

Si scorre l'esponente **in binario**. Per ogni bit, dal più significativo, si eleva al quadrato il risultato corrente; se il bit è 1 si moltiplica anche per la base. Ad ogni operazione si riduce modulo n. L'esponente 17 in binario è 10001.

Esempio svolto: 65^17 mod 3233

Si parte da 1. Il risultato corrente si indica con r.

Bit	Dopo il quadrato	Moltiplico per 65?	r
1	1	sì: 1 · 65 mod 3233	65
0	992	no	992
0	1232	no	1232
0	1547	no	1547
1	789	sì: 789 · 65 mod 3233	2790

Risultato: $65^{17} \bmod 3233 =$ `2790`. Sono bastati 5 quadrati e 2 moltiplicazioni invece di 16 moltiplicazioni: per esponenti grandi il risparmio è enorme (al più $2 \cdot \log_2 e$ operazioni).

In PHP

potenza-modulare.php

```php
<?php
function potenzaModulare(int $base, int $esp, int $mod): int {
    $risultato = 1;
    $base = $base % $mod;
    while ($esp > 0) {
        if ($esp & 1) {                      // bit meno significativo = 1?
            $risultato = ($risultato * $base) % $mod;
        }
        $base = ($base * $base) % $mod;      // quadrato
        $esp = $esp >> 1;                    // scarta il bit appena usato
    }
    return $risultato;
}

echo potenzaModulare(65, 17, 3233);
```

Il bit meno significativo dell'esponente si legge con `$esp & 1`; `$esp >> 1` scarta il bit usato. Con interi a 64 bit la funzione è corretta finché `$mod` è sotto circa 3 miliardi (il prodotto `$risultato * $base` deve stare in un intero). Per numeri più grandi servono BigInt o la libreria GMP.

Facile in un verso, difficile nell'altro

Moltiplicare due primi è immediato. Ritrovare i due primi partendo dal prodotto richiede molti tentativi.

n	Fattori	Tentativi di divisione
3233	53 · 61	27
1000036000099	1000003 · 1000033	500.002

Con numeri di 13 cifre (due primi di 7 cifre) la divisione per tentativi richiede già mezzo milione di passi; ogni cifra in più moltiplica il lavoro. Esistono algoritmi migliori, ma restano impraticabili per i numeri di RSA.

FUNZIONE UNIDIREZIONALE CON BOTOLA

Una funzione è **unidirezionale** se è facile da calcolare e difficile da invertire. È **con botola** se esiste un'informazione speciale che rende facile anche l'inversione. Dati p e q, calcolare n = p · q è facile; dato n, trovare p e q è difficile; ma chi conosce p e q ricava φ(n) e quindi d. Il segreto del proprietario della chiave è la botola.

DA RICORDARE

- **Modulo**: si lavora con i resti, e si può ridurre prima di calcolare.
- **MCD** con Euclide; **inverso** con Euclide esteso se MCD = 1.
- **$\varphi(p \cdot q) = (p - 1)(q - 1)$**; **$a^{\varphi(n)} \equiv 1$** per a coprimo con n.
- **Square-and-multiply** per le potenze; fattorizzare è il verso difficile.

Autoverifica

1. Calcola 250 mod 7, (−11) mod 6 e 2^10 mod 11 ricorrendo, dove serve, alla riduzione prima del calcolo.
2. Con l'algoritmo di Euclide calcola MCD(462, 1071) scrivendo tutte le divisioni.
3. Costruisci la tabella del prodotto modulo 9. Quali numeri hanno inverso? Perché gli altri no?
4. Con Euclide esteso trova l'inverso di 7 modulo 40 e verifica il risultato.
5. Calcola φ(35) e φ(77). Quale proprietà di 35 e 77 hai usato?
6. Calcola 7^13 mod 33 con square-and-multiply, scrivendo l'esponente in binario e la tabella dei passi.
7. Spiega perché per RSA si usa una funzione «con botola» e che cosa è la botola.

Per esercitarsi

- Slide del modulo 7 — con i calcolatori interattivi
- Simulazione: Euclide, MCD e inverso
- Simulazione: potenza modulare
- Scheda: esercizi di matematica per RSA — Euclide, inverso e potenze su carta
- Lab PHP 06: RSA giocattolo — mcd, inverso e potenza modulare in PHP

CAPITOLO 8

RSA

Il cifrario a chiave pubblica: chiavi, cifratura, firma digitale e perché non si rompe.

Il problema della chiave condivisa

Con un cifrario simmetrico mittente e destinatario hanno **la stessa chiave**. Ma come se la scambiano, se il canale non è sicuro? Ci sono due problemi: distribuire la chiave in modo sicuro, e gestire un numero di chiavi che cresce con il quadrato delle persone (n(n−1)/2, come si è visto nel capitolo 0: 4950 chiavi per 100 persone). La soluzione di RSA: due chiavi diverse, una pubblica e una privata.

Come nasce la chiave pubblica

Anno	Evento
1973	Clifford Cocks, al GCHQ britannico, ideò un sistema equivalente a RSA. Rimase segreto fino al 1997.
1976	Diffie e Hellman (con Merkle) pubblicano «New Directions in Cryptography»: scambio di chiavi su canale pubblico.
1977	Rivest, Shamir e Adleman, al MIT, inventano RSA. Descrizione formale nel 1978.
2000	Scade il brevetto statunitense di RSA: l'algoritmo diventa libero.

RSA sono le iniziali dei tre autori: Rivest, Shamir, Adleman. Diffie e Hellman propongono l'idea e un metodo per accordarsi su una chiave; RSA è il primo cifrario a chiave pubblica pubblicato che serve anche per cifrare e firmare.

Due chiavi: una pubblica, una privata

Bob genera le due chiavi e pubblica la chiave pubblica. Alice cifra il messaggio con la pubblica di Bob; il messaggio cifrato circola in chiaro sulla rete; solo Bob lo decifra con la sua privata.

- **Il lucchetto aperto**: la chiave pubblica è un lucchetto aperto. Chiunque può chiuderlo, solo Bob ha la chiave per riaprirlo.
- **Condizione**: dalla chiave pubblica non deve essere possibile ricavare quella privata (se non con un lavoro impraticabile).

Generare le chiavi in 5 passi

Si usano numeri piccoli per poter seguire i calcoli a mano. Nella realtà p e q hanno centinaia di cifre.

Passo	Operazione	Esempio
1	Scegli due numeri primi p e q (segreti, diversi)	p = 61, q = 53 (primi: true, true)
2	Calcola n = p · q (il «modulo», pubblico)	n = 61 · 53 = 3233
3	Calcola φ(n) = (p − 1)(q − 1) (segreto: serve conoscere p e q)	φ(n) = 60 · 52 = 3120
4	Scegli e con 1 < e < φ(n) e MCD(e, φ(n)) = 1 (nella pratica spesso 65537)	e = 17: MCD(17, 3120) = 1
5	Calcola d, inverso di e modulo φ(n) (Euclide esteso)	$d = 17^{-1} \bmod 3120 = 2753$

Verifica del passo 5: 17 · 2753 = 46801 = 15 · 3120 + 1. Il calcolo di d con la tabella di Euclide esteso è nel capitolo 7.

> **LE DUE CHIAVI**
> **Chiave pubblica**: (e, n) = (17, 3233). **Chiave privata**: (d, n) = (2753, 3233). Si buttano p, q e φ(n).

Cifrare e decifrare

Cifratura: $\mathbf{c = m^e \bmod n}$ Decifratura: $\mathbf{m = c^d \bmod n}$

Il messaggio m deve essere un numero minore di n. Per cifrare si usa solo la chiave pubblica; per decifrare serve la privata.

Esempio svolto: cifrare m = 65

c = 65^{17} mod 3233, con square-and-multiply (esponente 17 = `10001` in binario):

Bit	Dopo il quadrato	Moltiplico per 65?	r
1	1	sì	65
0	992	no	992
0	1232	no	1232
0	1547	no	1547
1	789	sì	2790

Il testo cifrato è `2790`.

Esempio svolto: decifrare c = 2790

m = 2790^{2753} mod 3233. L'esponente d = 2753 in binario è `101011000001`, 12 bit: bastano 12 quadrati e poche moltiplicazioni.

Bit	Dopo il quadrato	Moltiplico per 2790?	r
1	1	sì	2790
0	2269	no	2269
1	1425	sì	2393
0	806	no	806
1	3036	sì	3213
1	400	sì	615
0	3197	no	3197
0	1296	no	1296
0	1689	no	1689
0	1215	no	1215
0	1977	no	1977
1	3065	sì	65

Si ritrova `65`: il messaggio originale.

Perché funziona: l'intuizione di Eulero

Dal capitolo 7: se m è coprimo con n, allora $m^{\varphi(n)} \equiv 1 \pmod n$. Poiché d è l'inverso di e modulo φ(n), si ha $e \cdot d = 1 + k \cdot \varphi(n)$ per un certo intero k; qui $e \cdot d = 17 \cdot 2753 = 46801 = 1 + 15 \cdot 3120$. Allora:

$$(m^e)^d = m^{e \cdot d} = m^{1 + 15 \cdot \varphi} = m \cdot (m^{\varphi})^{15} \equiv m \cdot 1^{15} = \mathbf{m}$$

La decifratura annulla la cifratura: per questo d si sceglie come inverso di e modulo φ(n). Il caso (rarissimo) di m non coprimo con n funziona comunque: la dimostrazione completa usa il teorema cinese del resto.

Dalle parole ai numeri

RSA cifra **numeri** minori di n. Per le parole serve una codifica e dei blocchi: si associa a ogni lettera un numero di due cifre (A = 01, B = 02, … Z = 26) e si uniscono le lettere a gruppi. Con n = 3233 si possono unire due lettere: il blocco massimo è 2626 < 3233. La parola «CIAO» diventa i blocchi `0309 0115`, cioè i numeri `309, 115`.

Blocco m	Cifrato $c = m^{17} \bmod 3233$	Decifrato $c^{2753} \bmod 3233$
309	2851	309
115	1230	115

> **ERRORE TIPICO**
> Se n è piccolo (n ≤ 2626) si può usare un blocco di una sola lettera: RSA diventa allora una semplice sostituzione monoalfabetica, rompibile con le frequenze (capitolo 1).

La firma digitale: stesse chiavi, ruoli invertiti

Se si applica la chiave **privata** al messaggio, chiunque può controllare il risultato con la chiave **pubblica**. Solo chi possiede d può produrre una firma che la chiave pubblica verifica: è la prova dell'autore e dell'integrità.

Passo	Operazione	Chiave
Firma	s = m^d mod n	privata
Invio	(messaggio m, firma s)	
Verifica	s^e mod n	pubblica
Esito	se è uguale a m, la firma è autentica	

Firma di 65: 65^2753 mod 3233 = `588`. Verifica: 588^17 mod 3233 = `65`, che coincide con il messaggio. Se qualcuno modifica m, la verifica fallisce.

Si firma l'impronta, non il documento

Firmare direttamente un documento lungo richiederebbe molti blocchi RSA, lenti. Si firma invece una sua **impronta** (hash) di lunghezza fissa. Chi riceve ricalcola l'impronta, applica la chiave pubblica alla firma e confronta i due risultati (capitolo 9).

Chi può fare cosa

Obiettivo	Chi opera	Chiave usata	Chi controlla	Chiave usata
Riservatezza (cifrare)	Mittente	Pubblica del destinatario	Destinatario (decifra)	Privata del destinatario
Autenticità (firmare)	Autore	Privata dell'autore	Chiunque (verifica)	Pubblica dell'autore

Tutti possono cifrare per Bob e verificare le firme di Bob: serve solo la sua chiave pubblica. Solo Bob può decifrare i messaggi per Bob e firmare a nome di Bob: serve la sua chiave privata. Cifrare e firmare si possono combinare: firma con la propria privata e cifra con la pubblica del destinatario.

Sicurezza: fattorizzare n

L'attaccante vede n ed e. Se fattorizza n ricava φ(n), poi d, e legge tutto. Con n = 3233 è immediato: dividendo per tentativi si trova `53 · 61` dopo 27 tentativi. Da lì φ = 52 · 60 = 3120, e l'inverso di 17 modulo 3120 (Euclide esteso) è 2753: la chiave privata. Il codice PHP dell'attacco è alla fine del capitolo.

Con 2048 bit: un altro mondo

	Esempio	RSA-2048
Cifre di n	4	617
Divisioni per tentativi (circa √n/2)	27	circa 10^307

Anche a un miliardo di miliardi (10^18) di divisioni al secondo servirebbero circa 10^289 secondi. L'età dell'universo è circa 4·10^17 secondi. Esistono algoritmi molto più rapidi della divisione per tentativi, ma restano impraticabili a questa dimensione.

I record di fattorizzazione

Numero	Cifre	Bit	Fattorizzato
RSA-129	129	426	1994
RSA-768	232	768	2009
RSA-240	240	795	2019
RSA-250	250	829	2020
RSA-2048	617	2048	non fattorizzato

Per dare l'idea, la fattorizzazione di RSA-250 (2020) è costata circa 2700 anni-core di calcolo su tanti computer. Un RSA-2048 è molto più grande: la difficoltà cresce in modo rapidissimo con i bit. Fonti: sfide RSA Laboratories; annunci dei team di ricerca per RSA-240 e RSA-250.

Dimensioni consigliate

Modulo RSA	Sicurezza equivalente	Giudizio
1024 bit	80 bit	Non più accettabile
2048 bit	112 bit	Minimo attuale
3072 bit	128 bit	Consigliato per i nuovi sistemi
7680 bit	192 bit	Alta sicurezza
15360 bit	256 bit	Massima sicurezza (poco pratico)

Valori della raccomandazione NIST SP 800-57. «Sicurezza equivalente» indica un lavoro paragonabile alla forza bruta su una chiave simmetrica di quella lunghezza. Per avere la stessa sicurezza di AES a 128 bit serve una chiave RSA di 3072 bit: un motivo in più per usare RSA solo dove serve.

RSA «da libro» è insicuro

Se a ogni messaggio corrisponde sempre lo stesso cifrato (RSA **deterministico**), l'attaccante può provare tutti i messaggi possibili con la chiave pubblica e confrontare i risultati con il cifrato intercettato. Con messaggi di poche possibilità («sì» o «no») basta provarli tutti.

Problemi dell'RSA «da libro»	Rimedio: il padding
Deterministico: stessa m, stesso c	**OAEP** per cifrare: aggiunge casualità e struttura (RFC 8017)
Messaggi corti: se m^e < n basta una radice e-esima, senza modulo	**PSS** per firmare (RFC 8017)
Malleabile: dai cifrati si costruiscono altri cifrati validi	Si usa sempre una libreria collaudata (OpenSSL), mai codice scritto a mano

> **ATTENZIONE**
> Il nostro RSA giocattolo è ottimo per capire, ma non va mai usato per proteggere dati reali: numeri piccoli, nessun padding.

Il computer quantistico e l'algoritmo di Shor

Nel 1994 Peter Shor mostrò che un computer quantistico abbastanza grande potrebbe fattorizzare n in tempo polinomiale: RSA non reggerebbe. I computer quantistici attuali sono però lontanissimi dalle dimensioni richieste (le stime parlano di un numero enorme di qubit fisici corretti dagli errori): nessuno rompe oggi RSA-2048. Il NIST ha standardizzato nel 2024 i primi algoritmi di crittografia post-quantistica (FIPS 203, 204, 205) e promuove la transizione graduale; chi deve proteggere dati per molti anni inizia a pianificarla.

RSA e DES a confronto

	DES / AES (simmetrico)	RSA (asimmetrico)
Chiavi	Una sola, segreta, condivisa	Due: pubblica e privata
Problema principale	Scambiare la chiave	Lentezza e dimensione dei numeri
Velocità	Molto alta	Molto più bassa (ordini di grandezza)
Messaggi	Lunghi quanto serve	Brevi: minori di n
Usi tipici	Cifrare i dati	Scambiare chiavi, firmare
Sicurezza	Forza bruta sulla chiave	Fattorizzazione di n

I due sistemi si completano: RSA risolve lo scambio della chiave, il cifrario simmetrico fa il lavoro pesante. È la crittografia ibrida del capitolo 9: così funzionano HTTPS e SSH.

RSA giocattolo in PHP

Il programma usa soltanto interi, quindi funziona per numeri piccoli. La prima parte costruisce le chiavi (MCD, inverso con Euclide esteso, `generaChiavi()`). La seconda cifra, decifra e mostra l'attacco a n piccolo. Per verificarlo: `cifra(65, [17, 3233])` deve dare 2790; `decifra(2790, [2753, 3233])` deve restituire 65.

rsa-chiavi.php

```php
<?php
function mcd(int $a, int $b): int {
    while ($b !== 0) { [$a, $b] = [$b, $a % $b]; }
    return $a;
}

function inverso(int $e, int $phi): int {
    [$r0, $r1] = [$phi, $e];
    [$t0, $t1] = [0, 1];
    while ($r1 !== 0) {
        $q = intdiv($r0, $r1);
        [$r0, $r1] = [$r1, $r0 - $q * $r1];
        [$t0, $t1] = [$t1, $t0 - $q * $t1];
    }
    return (($t0 % $phi) + $phi) % $phi;   // t può essere negativo
}

function generaChiavi(int $p, int $q, int $e): array {
    $n = $p * $q;
    $phi = ($p - 1) * ($q - 1);
    if (mcd($e, $phi) !== 1) { throw new InvalidArgumentException('e non è coprimo con phi'); }
    $d = inverso($e, $phi);
    return ['pub' => [$e, $n], 'priv' => [$d, $n]];
}
```

rsa-uso.php

```php
<?php
// potenzaModulare() è quella del capitolo 7 (square-and-multiply)
function cifra(int $m, array $pub): int {
    return potenzaModulare($m, $pub[0], $pub[1]);      // c = m^e mod n
}

function decifra(int $c, array $priv): int {
    return potenzaModulare($c, $priv[0], $priv[1]);    // m = c^d mod n
}

function attacco(int $n, int $e): int {
    for ($p = 2; $p * $p <= $n; $p++) {                 // fattorizzazione ingenua
        if ($n % $p === 0) {
            $q = intdiv($n, $p);
            return inverso($e, ($p - 1) * ($q - 1));   // ricava d
        }
    }
    throw new InvalidArgumentException('n è primo: non è una chiave RSA');
}
```

DA RICORDARE

- **Chiavi**: $n = p \cdot q$, $\varphi = (p-1)(q-1)$, e coprimo con φ, $d = e^{-1} \bmod \varphi$.
- **Cifra** $c = m^e \bmod n$; **decifra** $m = c^d \bmod n$. Firma e verifica: ruoli invertiti.
- La sicurezza sta nella **difficoltà di fattorizzare n**; servono almeno 2048 bit e il padding.

Autoverifica

1. Con $p = 11$, $q = 13$ ed $e = 7$ calcola n, $\varphi(n)$ e d (con Euclide esteso). Verifica che $e \cdot d \equiv 1 \pmod{\varphi}$.
2. Con le chiavi dell'esercizio precedente cifra $m = 9$ e poi decifra il risultato. Quanti quadrati servono con square-and-multiply?
3. Perché per cifrare si usa la chiave pubblica del destinatario e per firmare la propria chiave privata? Che garanzia si ottiene nei due casi?
4. Se un attaccante conosce $n = 3233$ ed $e = 17$, che cosa gli manca per leggere i messaggi? Come può ottenerlo e perché non è fattibile con 2048 bit?
5. Perché m deve essere minore di n? Come si codifica una parola con $n = 3233$?
6. Che cos'è il padding OAEP e quale difetto dell'RSA «da libro» risolve?
7. Perché RSA non si usa per cifrare un intero file? Che cosa si fa in pratica?

Per esercitarsi

- Slide del modulo 8
- Simulatore RSA giocattolo — cifra, decifra, firma; anche da smartphone
- Gioco: RSA di classe — ogni studente genera la coppia di chiavi
- Scheda: RSA di classe — da stampare
- Lab PHP 06: RSA giocattolo — MCD, inverso, potenza modulare e attacco

CAPITOLO 9

Crittografia ibrida, firme e certificati

Come si mettono insieme cifrari simmetrici e RSA per costruire HTTPS, SSH e la firma digitale.

Due famiglie, due difetti

	Simmetrica (DES, AES)	Asimmetrica (RSA)
Chiavi	Una sola per cifrare e decifrare	Coppia: pubblica per cifrare, privata per decifrare
Velocità	Molto veloce, anche in hardware	Lenta e adatta solo a dati piccoli (meno di n)
Problema	Come si scambia la chiave in sicurezza? Per n persone servono n(n−1)/2 chiavi	Nessun segreto da scambiare prima; ogni utente ha una sola coppia

Persone	Chiavi simmetriche	Coppie RSA
10	45	10
100	4950	100
1000	499.500	1000

L'idea: RSA per la chiave, AES per i dati

La soluzione è usare i due sistemi insieme. Si genera una **chiave di sessione** casuale e usa-e-getta; RSA cifra soltanto quella chiave (pochi byte, quindi il costo è trascurabile); il cifrario simmetrico (AES, o DES nei nostri esempi didattici) cifra tutti i dati, con velocità di centinaia di MB al secondo su un PC. La chiave cifrata viaggia insieme ai dati cifrati.

Con RSA a 2048 bit e padding OAEP-SHA256 si possono cifrare al massimo 190 byte: la chiave AES (32 byte) ci sta di sicuro, un documento no.

Esempio svolto: Alice e Bob con numeri piccoli

Alice ha la coppia RSA dei capitoli precedenti: pubblica (e = 17, n = 3233), privata d = 2753. Bob vuole mandarle il messaggio «CIAO». Come cifrario simmetrico usiamo S-DES, che ha una chiave di 10 bit (quindi un numero minore di n).

Passo	Chi	Che cosa
1	Alice	Crea la coppia di chiavi RSA e pubblica la chiave pubblica (17, 3233).
2	Bob	Inventa una chiave di sessione casuale: 1010011010 (in decimale 666).
3	Bob	Cifra la chiave di sessione con la pubblica di Alice: c = 666^17 mod 3233 = 2892.
4	Bob	Cifra i dati con S-DES e la chiave di sessione: «CIAO» → 4FAA8424 (esadecimale).
5	Bob	Invia ad Alice la chiave cifrata (2892) e i dati cifrati (4FAA8424).
6	Alice	Con la sua privata ricava la chiave: 2892^2753 mod 3233 = 666 (cioè 1010011010).
7	Alice	Decifra i dati con la chiave di sessione e legge «CIAO».

Eve vede passare solo numeri senza senso: la chiave di sessione cifrata e i dati cifrati. Per leggerli dovrebbe rompere RSA o provare le chiavi del cifrario simmetrico.

Hash: l'impronta digitale di un dato

Una **funzione hash** trasforma un dato di qualunque lunghezza in un'impronta di lunghezza fissa. Le proprietà che servono sono tre:

- **Lunghezza fissa**: SHA-256 dà sempre 256 bit, qualunque sia il dato.
- **Un solo verso**: dall'impronta non si risale al testo.
- **Effetto valanga**: un carattere diverso cambia circa metà dei bit dell'impronta.

> **HASH GIOCATTOLO**
> Nei nostri esempi numerici usiamo un hash *giocattolo*: la somma dei codici dei caratteri modulo 256. Non è sicuro (serve solo a far tornare i conti con n = 3233). Per «PAGA 100» vale 202, per «PAGA 900» vale 210. Nella realtà si usa SHA-256.

La firma digitale: RSA al contrario

Per firmare si **cifra l'hash con la chiave privata**; chiunque verifica con la chiave pubblica.

Esempio svolto: firma di «PAGA 100»

1. Alice calcola l'hash del documento: «PAGA 100» → 202.
2. Alice cifra l'hash con la sua chiave privata (è la formula della decifratura RSA usata al contrario): firma = 202^2753 mod 3233 = `3218`. Invia documento e firma.
3. Bob ricalcola l'hash di «PAGA 100» (202) e applica la chiave pubblica alla firma: 3218^17 mod 3233 = `202`. I due numeri coincidono: **firma valida**.
4. Se Mallory cambia il documento in «PAGA 900», Bob ricalcola l'hash 210, ma la firma restituisce 202: non coincidono, **firma non valida**. Mallory non può rifare la firma perché non ha la chiave privata di Alice.

Che cosa garantisce una firma

Garanzia	Perché
Integrità	Se un bit del documento cambia, l'hash non coincide più
Autenticità	Solo chi ha la chiave privata poteva produrre quella firma
Non ripudio	Il firmatario non può negare, perché la privata è solo sua

> **CHE COSA NON FA**
> La firma non nasconde il contenuto (il documento resta leggibile) e non dice *chi è* il proprietario della chiave pubblica: lo dimostra solo se sei sicuro che quella pubblica sia davvero di Alice.

Di chi è questa chiave pubblica?

La chiave pubblica non è segreta, ma deve essere **autentica**. Un attaccante che si mette in mezzo (Mallory) può sostituirla: è l'attacco **man-in-the-middle**.

1. Alice chiede la chiave pubblica di Bob; Mallory intercetta la richiesta.
2. Mallory risponde con la *sua* chiave pubblica; Alice crede che sia di Bob e conserva la vera pubblica di Bob per sé.
3. Alice cifra la chiave di sessione con la pubblica sbagliata: apre Mallory con la sua privata e conosce la chiave.
4. Mallory ricifra la chiave con la vera pubblica di Bob e la inoltra: legge e può modificare tutto, e nessuno dei due se ne accorge.

Certificati e autorità di certificazione

Il rimedio è il **certificato** (standard X.509), un documento firmato che lega una chiave pubblica a un nome. Contiene:

- il nome del soggetto (per esempio `www.example.it`);
- la sua **chiave pubblica**;
- le date di inizio e fine validità;
- chi l'ha emesso;
- la **firma** dell'emittente.

Il certificato è firmato da una **autorità di certificazione** (CA). Le CA formano una **catena di fiducia**: la CA radice, le cui chiavi pubbliche sono già nel browser, firma una CA intermedia, che firma il certificato del sito. Il browser conosce in anticipo le chiavi pubbliche di poche CA radice: un certificato firmato da una CA fidata dimostra che quella chiave è del sito, e Mallory non può fabbricarne uno valido.

HTTPS: il protocollo TLS 1.3 in sintesi

TLS è il protocollo che protegge HTTPS (specifica: RFC 8446). Ecco la sintesi dell'handshake, cioè dello scambio iniziale:

Direzione	Messaggio
Client → Server	ClientHello: algoritmi supportati, numero casuale, parte pubblica Diffie-Hellman
Server → Client	ServerHello: algoritmo scelto + parte pubblica DH del server
Server → Client (cifrato)	Certificate + CertificateVerify (firma con la chiave del certificato) + Finished
Client → Server (cifrato)	Finished (il client ha verificato catena, nome e firma)
Client ⇄ Server (cifrato)	Dati HTTP cifrati con la chiave simmetrica derivata (AES-GCM o ChaCha20-Poly1305)

> **ATTENZIONE: RSA NON TRASPORTA PIÙ LA CHIAVE**
>
> Nelle vecchie versioni (TLS 1.2 con suite RSA) il client cifrava il segreto con la chiave RSA del server. In TLS 1.3 questo metodo non esiste più: il segreto nasce da Diffie-Hellman effimero. La chiave del certificato (RSA o curve ellittiche) serve soltanto a **firmare** e provare l'identità.

Diffie-Hellman: accordarsi senza trasmettere

Diffie-Hellman permette ad Alice e Bob di costruire lo stesso segreto parlando in pubblico. Con numeri piccoli: p = 23 (primo) e g = 5 sono pubblici.

Passo	Alice	Bob
1. Segreto	sceglie a = 6	sceglie b = 15
2. Parte pubblica	A = g^a mod p = 5^6 mod 23 = 8	B = g^b mod p = 5^15 mod 23 = 19
3. Si scambiano A e B	riceve B = 19	riceve A = 8
4. Segreto comune	B^a mod p = 19^6 mod 23 = 2	A^b mod p = 8^15 mod 23 = 2

Il segreto comune è `2`, mai trasmesso. Eve conosce 23, 5, 8 e 19, ma per trovare il segreto dovrebbe risolvere il *logaritmo discreto*, difficile con numeri grandi. Il valore è **effimero**: per ogni connessione si generano numeri nuovi e si buttano, quindi se il server viene violato in futuro le sessioni passate restano al sicuro (*forward secrecy*). Diffie-Hellman da solo non autentica: un Mallory in mezzo farebbe due scambi. Per questo in TLS la parte del server è firmata e il certificato la lega al nome del sito.

Che cosa c'è dietro il lucchetto

- catena di certificati fino a una radice fidata;
- nome del sito uguale a quello del certificato;
- date di validità e revoca;
- firma sull'handshake (CertificateVerify).

Un esempio reale, ottenibile con `openssl s_client`: protocollo TLSv1.3, suite TLS_AES_256_GCM_SHA384, chiave del server ECDSA a 256 bit, scambio X25519 effimero. I siti moderni spesso usano certificati con **curve ellittiche** (ECDSA), con chiavi di 256 bit equivalenti a un RSA molto più grande: la logica (firma e verifica) è la stessa vista con RSA.

SSH: chiavi in ~/.ssh e known_hosts

terminale

```
# genera una coppia di chiavi (moderna: ed25519)
ssh-keygen -t ed25519 -C "thomas@scuola"

# cartella ~/.ssh
id_ed25519       # chiave PRIVATA: mai condividerla
id_ed25519.pub   # chiave pubblica: la metti sul server
known_hosts      # chiavi dei server già incontrati
authorized_keys  # (sul server) chiavi utente ammesse
```

- **Server → utente**: al primo accesso il client mostra l'impronta della chiave del server e la salva in `known_hosts`. Dalle volte successive la confronta.
- **Utente → server**: il server sfida il client, che firma con la chiave privata. La privata non viaggia mai.

Se la chiave del server cambia, il client lo segnala con un avviso: può essere una reinstallazione, ma anche un attacco man-in-the-middle.

se la chiave del server cambia

```
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@    WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!     @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
```

Email cifrata e firma digitale qualificata

- **PGP / OpenPGP e S/MIME**: schema ibrido (una chiave di sessione cifra il testo, la chiave pubblica del destinatario cifra la chiave di sessione) e firma sull'hash del messaggio. Fiducia: in PGP si verificano le impronte (*web of trust*), in S/MIME si usano certificati emessi da CA.
- **Firma digitale qualificata** (Italia): certificato qualificato di un prestatore di servizi fiduciari e dispositivo sicuro (smart card, token o firma remota). Per il regolamento europeo eIDAS ha il valore giuridico della firma autografa. Formati comuni: .p7m (CAdES) e PDF firmati (PAdES).
- **SPID e CIE** servono soprattutto a *identificarsi* online, cioè autenticarsi, e non sono di per sé una firma qualificata.

Mappa: chi fa che cosa nei protocolli

Protocollo	Accordo sulla chiave	Identità / firma	Dati
HTTPS (TLS 1.3)	Diffie-Hellman effimero (X25519)	Certificato (RSA o ECDSA) + firma	AES-GCM, ChaCha20-Poly1305
SSH	Diffie-Hellman (curve25519)	Chiave del server e chiave utente	AES o ChaCha20
Email OpenPGP	Chiave di sessione cifrata con RSA / ECC	Firma sull'hash	AES
Firma qualificata	—	RSA o ECDSA sull'hash (SHA-256)	documento in chiaro + firma

Dai bigliettini di Cesare a HTTPS

Il percorso del corso si legge come una sequenza di rimedi: ogni passo corregge una debolezza del precedente.

- Troppe poche chiavi (Cesare: 25) → chiavi più lunghe (Vigenère, trasposizione).
- Lettere con frequenze visibili → bit e diffusione (DES, AES).
- Chiave da scambiare in segreto → coppia di chiavi (RSA).
- Chiave pubblica di cui fidarsi → certificati.

> **PRINCIPIO DI KERCKHOFFS**
> Gli algoritmi sono pubblici e studiati da tutti; la sicurezza sta solo nelle chiavi, che non si inventano: si generano casuali con strumenti collaudati.

Cifrario	Tipo	Chiave	Debolezza
Cesare	sostituzione	1 numero (25 utili)	forza bruta, frequenze
Cesare a chiave numerica	polialfabetico	serie di numeri	chiave ripetuta: colonne di Cesare
Vigenère	polialfabetico	una parola	chiave ripetuta: Kasiski, indice di coincidenza
Trasposizione	permutazione	ordine delle colonne	le lettere restano quelle: anagrammi
XOR con chiave ripetuta	a flusso	bit ripetuti	testo noto, chiave riusata
S-DES	a blocchi (didattico)	10 bit	solo 1024 chiavi
DES / 3DES	a blocchi 64 bit	56 bit / fino a 168	DES: forza bruta; 3DES: lento, blocco piccolo
AES	a blocchi 128 bit	128-256 bit	nessuna nota in pratica; ECB espone i pattern
RSA	asimmetrico	coppia, $n = p \cdot q$	se n si fattorizza; serve ≥ 2048 bit e padding

Dal foglio al terminale: la traccia del laboratorio OpenSSL

Con OpenSSL si ripete al computer, con strumenti veri, quanto visto con numeri piccoli. Questa è la sequenza essenziale (la scheda del laboratorio la guida passo per passo, con il confronto tra ECB e CBC, il tentativo di cifrare un file con RSA, i certificati dei siti con `openssl s_client`).

openssl.sh

```
# 1. Alice crea la coppia di chiavi RSA (una volta)
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out alice_priv.pem
openssl pkey -in alice_priv.pem -pubout -out alice_pub.pem

# 2. Bob cifra il documento con una chiave di sessione casuale (AES)
openssl rand -hex 32 > sessione.key
openssl enc -aes-256-cbc -pbkdf2 -in documento.txt -out documento.enc -pass file:sessione.key

# 3. Bob cifra la sola chiave di sessione con la pubblica di Alice (RSA)
openssl pkeyutl -encrypt -pubin -inkey alice_pub.pem -pkeyopt rsa_padding_mode:oaep -in
sessione.key -out sessione.enc

# 4. Alice recupera la chiave con la sua privata e decifra il documento
openssl pkeyutl -decrypt -inkey alice_priv.pem -pkeyopt rsa_padding_mode:oaep -in sessione.enc -
out sessione.key
openssl enc -d -aes-256-cbc -pbkdf2 -in documento.enc -pass file:sessione.key

# 5. Firma e verifica (SHA-256 e RSA)
openssl dgst -sha256 -sign alice_priv.pem -out documento.sig documento.txt
openssl dgst -sha256 -verify alice_pub.pem -signature documento.sig documento.txt
```

NOTA

Si cifra con RSA la sola chiave di sessione; il documento, che è molto più grande di n, viaggia cifrato con AES. Provando a cifrare direttamente un file grande con RSA, OpenSSL risponde con un errore: il dato supera la dimensione massima. È la dimostrazione pratica del perché esiste la crittografia ibrida.

DA RICORDARE

- La **simmetrica** è veloce ma serve una chiave condivisa; l'**asimmetrica** è lenta ma risolve lo scambio: si usano insieme (**schema ibrido**).
- L'**hash** dà un'impronta; la **firma** la cifra con la privata e garantisce integrità e autenticità.
- I **certificati** legano una chiave pubblica a un nome e fermano il man-in-the-middle.
- **TLS 1.3** usa Diffie-Hellman effimero per la chiave e la firma per l'identità; **SSH** segue la stessa logica.

Autoverifica

1. Perché non si cifra un intero file con RSA? Descrivi i passaggi della cifratura ibrida.
2. Che cosa si firma in una firma digitale e perché? Chi usa quale chiave per firmare e per verificare?
3. Descrivi l'attacco man-in-the-middle sulla chiave pubblica e spiega come i certificati lo bloccano.
4. Con p = 23, g = 5, a = 4 e b = 3 calcola A, B e il segreto condiviso di Diffie-Hellman.
5. Che cosa significa «effimero» in Diffie-Hellman effimero e quale proprietà garantisce?
6. Che differenza c'è tra una firma digitale qualificata e l'accesso con SPID o CIE?
7. Collega ogni debolezza della tabella finale al cifrario che la corregge.

Per esercitarsi

- Slide del modulo 9
- Simulazione: crittografia ibrida — con la tua chiave e il tuo messaggio
- Lab OpenSSL — simmetrica, asimmetrica, ibrida, firme e certificati dei siti

APPENDICE A

Glossario

AES
Advanced Encryption Standard. Cifrario a blocchi simmetrico a 128 bit con chiave da 128, 192 o 256 bit, standard dal 2001 (FIPS 197).

Algoritmo (cifrario)
La regola con cui si trasforma il testo in chiaro in testo cifrato. Per il principio di Kerckhoffs può essere pubblica.

Asimmetrica (crittografia)
Sistema con una coppia di chiavi legate: una pubblica e una privata. Ciò che una cifra, solo l'altra decifra.

Attacco a testo noto
Attacco in cui l'attaccante conosce una parte del testo in chiaro e il corrispondente cifrato (per esempio l'inizio di un messaggio).

Autorità di certificazione (CA)
Ente che emette certificati firmandoli; le CA radice sono già presenti nel browser.

Blocco
Gruppo di bit di lunghezza fissa su cui lavora un cifrario a blocchi (64 bit nel DES, 128 nell'AES).

CBC (Cipher Block Chaining)
Modalità operativa in cui ogni blocco in chiaro è combinato con XOR con il cifrato del blocco precedente prima di essere cifrato.

Certificato (X.509)
Documento firmato che lega una chiave pubblica a un nome (soggetto), con date di validità ed emittente.

Chiave
Il dato segreto che governa il funzionamento di un algoritmo di cifratura.

Cifrare / decifrare
Trasformare un testo in chiaro in testo cifrato (cifrare) e viceversa conoscendo la chiave (decifrare). Da non usare «criptare».

Confusione
Proprietà di Shannon: il legame tra chiave e cifrato deve essere complicato. Si ottiene con la sostituzione (S-box).

Congruenza
$a \equiv b \pmod n$ significa che a e b hanno lo stesso resto nella divisione per n.

Coprimi (numeri)
Due numeri senza fattori in comune, cioè con MCD uguale a 1.

Crittoanalisi
Studio dei metodi per leggere o rompere un messaggio cifrato senza conoscere la chiave.

DES
Data Encryption Standard. Cifrario a blocchi di 64 bit con chiave di 56 bit e 16 round di Feistel, standard dal 1977 (FIPS 46-3).

Diffie-Hellman
Metodo con cui due parti costruiscono uno stesso segreto comunicando su un canale pubblico.

Diffusione
Proprietà di Shannon: ogni bit del messaggio deve influenzare molti bit del cifrato. Si ottiene con permutazioni e più round.

ECB (Electronic Codebook)
Modalità operativa in cui ogni blocco si cifra da solo: blocchi uguali producono cifrati uguali.

Effetto valanga
Proprietà per cui cambiando un solo bit in ingresso cambia circa metà dei bit in uscita.

Euclide (algoritmo di)
Metodo per calcolare il MCD con divisioni successive; nella versione estesa fornisce anche l'inverso modulare.

Feistel (rete di)
Struttura a round che divide il blocco in due metà L e R: $L' = R$, $R' = L \oplus F(R, K)$. Si decifra con le sottochiavi in ordine inverso.

Firma digitale
Hash del documento cifrato con la chiave privata del firmatario; si verifica con la chiave pubblica.

Forward secrecy
Proprietà per cui la violazione futura di una chiave non permette di decifrare le sessioni passate; ottenuta con Diffie-Hellman effimero.

Forza bruta
Attacco che prova tutte le chiavi possibili.

Frequenze (analisi delle)
Attacco che confronta la frequenza delle lettere del cifrato con quella del linguaggio.

Funzione con botola
Funzione facile da calcolare, difficile da invertire, ma facile da invertire per chi conosce un'informazione segreta (la botola).

Hash
Funzione che associa a un dato qualsiasi un'impronta di lunghezza fissa, non invertibile e molto sensibile alle modifiche (per esempio SHA-256).

Ibrida (crittografia)
Uso combinato di crittografia asimmetrica per scambiare una chiave e simmetrica per cifrare i dati.

Indice di coincidenza (IC)
Probabilità che due lettere prese a caso in un testo siano uguali: circa 0,074 per l'italiano, circa 0,038 per lettere casuali.

Inverso modulare
Per a modulo n, il numero x con a·x ≡ 1 (mod n); esiste se e solo se MCD(a, n) = 1.

IV (vettore di inizializzazione)
Valore casuale, non segreto, usato per cifrare il primo blocco in CBC; va cambiato a ogni messaggio.

Kasiski (metodo di)
Stima della lunghezza della chiave di Vigenère dalle distanze tra sequenze ripetute nel cifrato.

Kerckhoffs (principio di)
La sicurezza di un cifrario deve dipendere solo dalla segretezza della chiave e non da quella dell'algoritmo.

Man-in-the-middle
Attacco in cui l'attaccante si interpone tra le parti, sostituendo per esempio una chiave pubblica con la propria.

MCD
Massimo comun divisore: il più grande numero che divide due numeri dati.

Modulo (aritmetica modulare)
Aritmetica in cui i numeri «girano» come su un orologio: a mod n è il resto della divisione di a per n.

Monoalfabetico / polialfabetico
Cifrario che usa un solo alfabeto di sostituzione (Cesare) oppure più alfabeti in rotazione (Vigenère).

One-time pad
Cifrario XOR con chiave casuale, lunga quanto il messaggio e usata una sola volta: sicurezza perfetta.

Padding
Riempimento aggiunto a un messaggio. In RSA (OAEP, PSS) aggiunge casualità e struttura; in un cifrario a blocchi completa l'ultimo blocco.

Potenza modulare
Calcolo di m^e mod n; con square-and-multiply si fa riducendo modulo n a ogni passo.

RSA
Cifrario asimmetrico di Rivest, Shamir e Adleman (1977) basato sulla difficoltà di fattorizzare n = p·q.

S-box
Tabella di sostituzione non lineare: fonte di confusione nei cifrari a blocchi (8 S-box da 6 a 4 bit nel DES).

S-DES
Versione semplificata del DES a scopo didattico (blocco 8 bit, chiave 10 bit, 2 round).

Simmetrica (crittografia)
Sistema in cui cifratura e decifratura usano la stessa chiave segreta.

Sostituzione / trasposizione
Le due operazioni elementari dei cifrari: cambiare le lettere o cambiarne l'ordine.

Square-and-multiply
Algoritmo che scorre l'esponente in binario elevando al quadrato e moltiplicando quando il bit è 1.

Steganografia
Tecnica che nasconde l'esistenza stessa di un messaggio.

TLS
Transport Layer Security: protocollo che protegge HTTPS; la versione 1.3 è descritta nella RFC 8446.

Triple DES (3DES)
DES applicato tre volte (cifra-decifra-cifra) con due o tre chiavi; sicurezza effettiva circa 112 bit.

Vigenère (cifrario di)
Cifrario polialfabetico con chiave a parola: ogni lettera si sposta del valore della lettera corrispondente della chiave.

XOR (⊕)
OR esclusivo: dà 1 se i due bit sono diversi. $A \oplus K \oplus K = A$.

φ di Eulero
$\varphi(n)$ è il numero di interi tra 1 e n coprimi con n; $\varphi(p \cdot q) = (p - 1)(q - 1)$ per p e q primi.

APPENDICE B

Fonti e letture

Riferimenti essenziali. Gli esempi numerici della dispensa sono calcolati con il motore del progetto (`assets/crypto.js`) e verificati sui vettori di prova ufficiali: per il DES, chiave 133457799BBCDFF1 e blocco 0123456789ABCDEF con cifrato atteso 85E813540F0AB405; per RSA, l'esempio a mano con p = 61, q = 53, e = 17.

Standard e specifiche

- NIST, *FIPS PUB 46-3: Data Encryption Standard (DES)*, 25 ottobre 1999 (ritirato nel 2005).
- NIST, *FIPS PUB 197: Advanced Encryption Standard (AES)*, 2001 (aggiornato nel 2023).
- K. Moriarty, B. Kaliski, J. Jonsson, A. Rusch, *RFC 8017: PKCS #1: RSA Cryptography Specifications Version 2.2*, IETF, novembre 2016.
- E. Rescorla, *RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3*, IETF, agosto 2018.
- D. Cooper et al., *RFC 5280: Internet X.509 Public Key Infrastructure Certificate and CRL Profile*, IETF, maggio 2008.
- E. Barker, *NIST SP 800-57 Part 1: Recommendation for Key Management*, NIST (raccomandazioni sulle dimensioni delle chiavi).
- NIST, *FIPS 203, 204, 205* (algoritmi di crittografia post-quantistica), agosto 2024.

Libri

- W. Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson (varie edizioni). Fonte dell'esempio S-DES.
- S. Singh, *Codici & segreti*, Rizzoli (edizione originale: *The Code Book*, 1999). Storia della crittografia dai cifrari antichi a RSA.
- Electronic Frontier Foundation, *Cracking DES: Secrets of Encryption Research, Wiretap Politics and Chip Design*, O'Reilly, 1998 (la macchina «Deep Crack»).

Articoli originali

- A. Kerckhoffs, «La cryptographie militaire», *Journal des sciences militaires*, 1883.
- F. W. Kasiski, *Die Geheimschriften und die Dechiffrir-Kunst*, 1863.
- C. E. Shannon, «Communication Theory of Secrecy Systems», *Bell System Technical Journal*, 1949.
- W. Diffie, M. E. Hellman, «New Directions in Cryptography», *IEEE Transactions on Information Theory*, 1976.
- R. L. Rivest, A. Shamir, L. Adleman, «A Method for Obtaining Digital Signatures and Public-Key Cryptosystems», *Communications of the ACM*, 1978.